Dossier Projet

Tabletop Games Management Tool

Titre RNCP Développeur Web et Web Mobile

kevin viais

26/02/2025

Table des matières

Remerciements 4

Introduction 5

Compétences du référentiel 6

1. Développer la partie front-end d'une application web ou web mobile de façon sécurisé 6

   a. Maquetter des interfaces utilisateur web ou web mobile 6

   b. Réaliser des interfaces utilisateur statiques web ou web mobile 6

   c. Développer la partie dynamique des interfaces utilisateurs web ou web mobile 6

2. Développer la partie back-end d'une application web ou web mobile de façon sécurisée 6

   a. Mettre en place une base de donnée relationnelle 6

   b. Développer des composant d'accès au données SQL et NoSQL 6

   c. Développer des composant métier coté server 6

Besoins du projet 7

1. Contexte 7

2. Cahier des charges 7

   a. Besoins initiaux: 7

   b. Besoins fonctionnels : 8

   c. Besoins non fonctionnels : 10

3. Contraintes et livrables attendus 11

Gestion du projet 12

1. Environnement humain et technique 12

2. Objectifs qualité 13

Réalisations front-end 15

1. Organisation du code et présentation générale d'Angular 16

a . Composants : 16

b. Services : 19

c. Modules : 21

d. Modèles : 21

e. Routing : 22

f. Guards : 23

g. Directives et pipes : 25

2. Organisation de l'interface 26

a. Maquettes : 26

b. Adaptation Web et Web Mobile (Responsive Design) 26

c. Site map 28

3. Interface utilisateur – présentation générale et exemples. 29

a. Éléments communs et pages statiques 29

b. Liste des jeux (game-card.component) 31

c. Page d'un jeu (single-game.component) 32

d. Connexion et inscription (sign-in, sign-up et reset-password.component) 33

e. Liste et création de tables (pour utilisateur connecté) 34

f. Le dashboard 35

4. Optimisation pour les moteurs de recherche (SEO) et accessibilité 37

a. Optimisation pour les moteurs de recherche (SEO) : 37

b. Accessibilité : 38

Réalisation back-end 40

1. Structure du code 41

2. Base de données 42

3. API RESTFul 44

4. Composants métier : 45
a. gameController.js : 45
b. tableController.js : 46
c. userController.js : 47
d. openingHoursController.js : 49
Eléments de sécurité 50
1. Authentification et autorisation : 50
2. Protection contre les failles XSS : 50
3. Validation des données contre injection SQL (et NoSQL): 51
4. Stockage sécurisé des informations sensibles : 51
Jeux d'essai 52
Respect Du RGPD 52
Déploiement : 54
Veille durant le projet 55
Conclusion 56
Annexes 57
Annexe 1 : Diagramme des cas d'utilisation 58
Annexe 2 : Liste des librairies 59
Annexe 3 : Liste des composants Angular 61
Annexe 4 : Maquettes 63
Annexe 5 : tableau de l'API REST 66
Annexe 7 : Schéma de BDD et Diagramme de classes UML 70
Annexe 8 : Jeu d'essais 72

Remerciements

La réalisation de ce projet n'aurait pas été possible sans le soutien et l'aide de plusieurs personnes, que je tiens à remercier chaleureusement.

Je souhaite tout d'abord exprimer ma gratitude à l'équipe pédagogique de Holberton School, pour leur enseignement de qualité, leur disponibilité et leurs conseils avisés tout au long de cette formation. Leur expertise et leur passion pour le développement web ont été une source d'inspiration constante.

Je remercie particulièrement Sarah Benamor, pour son accompagnement personnalisé, son écoute attentive, et ses retours constructifs, qui m'ont permis de progresser et de mener ce projet à bien.

Je remercie également l'équipe du bar à jeux "Ô p'tit bonheur la chance", et en particulier Elise Quetstroey, pour leur accueil chaleureux, leur disponibilité, et pour m'avoir permis de comprendre les besoins concrets d'un établissement de ce type. Leurs retours et leurs suggestions ont été précieux pour orienter la conception de l'application.

Je remercie mes camarades de promotion, amis, et famille, pour leur soutient, leurs relectures attentives du code et du dossier et leurs conseils avisés sur l'interface utilisateur, ainsi que pour l'émulation collective qui a régné tout au long de cette formation.

Je tiens également à remercier la communauté des développeurs Angular et Node.js, dont les nombreuses contributions en ligne (blogs, forums, tutoriels) m'ont été d'une grande aide

Enfin, je remercie le jury de soutenance pour l'attention qu'il portera à ce projet et pour ses questions, qui me permettront, je l'espère, de progresser encore.

Introduction

Le secteur des jeux de société connaît depuis quelques années un engouement croissant, porté par une offre de jeux de plus en plus diversifiée et par l'émergence de lieux conviviaux dédiés à cette pratique : les bars à jeux. Ces établissements, qui proposent à leurs clients de découvrir et de jouer à des jeux de société tout en consommant des boissons et des repas, rencontrent un succès important. Cependant, la gestion des réservations de tables, en particulier lors des pics de fréquentation, peut s'avérer complexe et chronophage, tant pour le personnel de l'établissement que pour les clients.

C'est dans ce contexte qu'est né le projet *Tabletop Games Meeting Tool* (TGMT). L'objectif principal de ce projet, réalisé dans le cadre de ma formation « Les fondamentaux du développement » chez Holberton School pour l'obtention du titre RNCP de niveau 5 Développeur Web et Web Mobile, était de développer une application web modulaire offrant des services supplémentaires, capable de s'intégrer au site web existant d'un bar à jeux, permettant de simplifier et d'optimiser la réservation de tables de jeux au sein d'un tel établissement. Ce projet a été initialement conçu en collaboration avec un bar à jeux local, mais il est désormais développé dans une optique d'adaptabilité à n'importe quel bar à jeux souhaitant adopter une solution similaire.

L'application TGMT est une application modulaire qui s'intègre facilement à des sites existants. Elle offre aux clients une interface intuitive pour consulter la liste des jeux disponibles, visualiser les détails de chaque jeu, et réserver une table pour un jeu à une date et une heure spécifiques. Elle propose également aux administrateurs du bar à jeux une interface de gestion complète pour ajouter, modifier ou supprimer des jeux, gérer les tables (création, modification, suppression) et gérer les utilisateurs.

Pour répondre aux exigences de performance, de sécurité et de maintenabilité, l'application a été développée en utilisant une architecture moderne, séparant clairement le frontend et le backend. Le frontend a été réalisé avec le framework Angular, tandis que le backend repose sur Node.js et Express, avec une base de données MongoDB pour le stockage des données. L'authentification des utilisateurs est gérée via des tokens JWT. L'application est hébergée sur la plateforme Render.

Après rappel des compétences du référentiel, ce dossier présentera, dans un premier temps, la phase de conception du projet, en détaillant les besoins utilisateurs, les fonctionnalités implémentées, l'architecture technique retenue et le modèle de données. Dans un second temps, il exposera les aspects liés au développement frontend (avec Angular) et backend (avec Node.js/Express et MongoDB), en mettant l'accent sur les choix d'implémentation et les points de sécurité. Enfin, nous aborderons les tests effectués, le déploiement de l'application, et conclurons sur le bilan du projet et les évolutions futures envisagées.

Compétences du référentiel

1. Développer la partie front-end d'une application web ou web mobile de façon sécurisé
    a. Maquetter des interfaces utilisateur web ou web mobile
    b. Réaliser des interfaces utilisateur statiques web ou web mobile
    c. Développer la partie dynamique des interfaces utilisateurs web ou web mobile
2. Développer la partie back-end d’une application web ou web mobile de façon sécurisée
    a. Mettre en place une base de donnée relationnelle
    b. Développer des composant d’accès au données SQL et NoSQL
    c. Développer des composant métier coté server

Besoins du projet

1. Contexte

Le projet *Tabletop Games Meeting Tool* (TGMT) s'inscrit dans le contexte de l'essor des bars à jeux, des établissements qui combinent la convivialité d'un bar traditionnel avec le plaisir du jeu de société. L'application a été initialement inspirée par les besoins du bar à jeux "Ô p'tit bonheur la chance", situé à Bons-en-Chablais. Cet établissement vise une ambiance cosy et s'adresse à une clientèle variée, allant des familles aux joueurs experts, en passant par les groupes d'amis et même les institutions. L'offre ludique y est large, couvrant tous les types de jeux de société : jeux de plateau, jeux de cartes, jeux de rôle, jeux de figurines, jeux traditionnels et jeux d'ambiance. Bien que le bar propose également une petite restauration et des boissons, l'application TGMT se concentre exclusivement sur la gestion des jeux et la réservation de tables.

2. Cahier des charges

a. Besoins initiaux:

Avant la mise en place de TGMT, le bar à jeux rencontrait plusieurs difficultés :

- **Déplacement des utilisateurs :** Les clients potentiels devaient se déplacer physiquement jusqu'au bar, ou téléphoner, pour consulter la liste des jeux disponibles, s'assurer de la présence d'un jeu spécifique, ou connaître les disponibilités des tables. Cela représentait un frein pour une partie de la clientèle, et une perte de temps pour le personnel.
- **Gestion manuelle des réservations :** Les réservations étaient gérées manuellement (par téléphone ou sur place), ce qui était source d'erreurs (doubles réservations, oublis), de perte de temps pour le personnel, et d'une expérience client potentiellement négative (incertitude, attente).
- **Manque de visibilité sur l'offre :** Même si la liste des jeux était disponible, il était difficile pour les clients d'avoir une vue d'ensemble claire et détaillée de l'offre ludique du bar (informations sur les jeux, règles, ressources, etc.).

- **Manque de visibilité de l'affluence:** Le personnel n'avait pas de moyen de savoir à l'avance le nombre de personnes attendues.
- **Déplacements inutiles et absence de partenaires de jeu :** Un joueur pouvait se déplacer jusqu'au bar, attiré par un jeu spécifique ou l'envie de découvrir de nouveaux jeux, et se retrouver sans partenaire de jeu disponible, rendant sa venue infructueuse.

b. Besoins fonctionnels :

Pour répondre à ces problèmes, l'application TGMT devait offrir les fonctionnalités suivantes, réparties entre les différents types d'utilisateurs :

Pour les joueurs (Utilisateurs standards) :

- **Consultation du catalogue de jeux :**
  - Pouvoir consulter la liste complète des jeux disponibles, avec une présentation claire et attractive (image, nom, type, catégorie).
  - Pouvoir rechercher un jeu par nom.
  - Pouvoir filtrer les jeux par type, catégorie, âge minimum, difficulté, auteur, nombre de joueurs, et durée de partie.
  - Pouvoir accéder à une page de détails pour chaque jeu, contenant les informations suivantes : description, règles (lien externe), liens vers des ressources (vidéos, etc.), et la liste des tables de jeu disponibles pour ce jeu.
- **Réservation de tables :**
  - Pouvoir visualiser les tables de jeu disponibles pour un jeu donné, avec leurs horaires de début et de fin, leur statut (ouverte/fermée), le nombre de participants inscrits et le nombre maximum de participants.
  - Pouvoir s'inscrire à une table de jeu ouverte (si le nombre maximum de participants n'est pas atteint).
  - Pouvoir se désinscrire d'une table de jeu.

- **Gestion du compte utilisateur :**
  - Pouvoir créer un compte utilisateur.
  - Pouvoir se connecter et se déconnecter.
  - Pouvoir consulter et modifier ses informations personnelles (nom d'utilisateur, prénom, nom, email, téléphone).
  - Pouvoir gérer une liste de jeux favoris (ajouter/supprimer).
  - Pouvoir consulter la liste de ses jeux favoris.
  - Pouvoir réinitialiser son mot de passe en cas d'oubli.

Pour les administrateurs (Personnel du bar) :

- **Gestion des jeux :**
  - Pouvoir ajouter un nouveau jeu au catalogue (avec toutes les informations nécessaires, y compris une image).
  - Pouvoir modifier les informations d'un jeu existant.
  - Pouvoir supprimer un jeu du catalogue.
- **Gestion des tables :**
  - Pouvoir créer une nouvelle table de jeu, en l'associant à un jeu existant, en définissant une date et heure de début, une durée, et un nombre maximum de participants.
  - Pouvoir modifier les informations d'une table de jeu existante.
  - Pouvoir supprimer une table de jeu.
- **Gestion des utilisateurs :**
  - Pouvoir consulter la liste de tous les utilisateurs.
  - Pouvoir modifier le rôle d'un utilisateur (administrateur/utilisateur standard).
  - Pouvoir supprimer un utilisateur.

- **Gestion des horaires :**
  - Pouvoir définir et modifier les horaires d'ouverture du bar, pour chaque jour de la semaine.

c. Besoins non fonctionnels :

- **Performance :** L'application doit être réactive, avec des temps de chargement rapides (moins de 3 secondes pour la plupart des pages).

- **Sécurité :**
  - L'application doit protéger les données des utilisateurs (en particulier les mots de passe, qui doivent être hachés).
  - L'accès aux fonctionnalités d'administration doit être strictement réservé aux utilisateurs autorisés.
  - L'application doit être protégée contre les attaques courantes (XSS, injections SQL).

- **Accessibilité :** L'application doit être responsive et s'adapter aux différents types de terminaux (ordinateurs, tablettes, smartphones).
- **Utilisabilité :** L'application doit être intuitive et facile à utiliser, avec une interface claire et une navigation simple.
- **Maintenabilité :** Le code doit être clair, commenté, et bien structuré, pour faciliter la maintenance et les évolutions futures.
- **Évolutivité :** L'architecture de l'application doit permettre d'ajouter facilement de nouvelles fonctionnalités.
- **RGPD :** L'application doit être conforme au Règlement Général sur la Protection des Données (RGPD). Les données personnelles collectées doivent être limitées au strict nécessaire, et les utilisateurs doivent être informés de l'utilisation de leurs données.
- **Cookies :** L'application ne doit utiliser que des cookies strictement nécessaires à son fonctionnement. Aucune donnée personnelle ne doit être revendue ou utilisée à des fins statistiques non anonymisées.

Le diagramme des cas d’utilisation est disponible en annexe (1).

3. Contraintes et livrables attendus

Le développement de l'application TGMT s'est inscrit dans le cadre d'une formation intensive de développeur web et web mobile, imposant un calendrier serré avec une soutenance finale. Le projet devait être mené à bien en 8 semaines.

Bien qu'une certaine souplesse ait été accordée concernant le choix des technologies, l'utilisation de frameworks modernes et largement répandus dans l'industrie, tels qu'Angular pour le frontend et Node.js/Express pour le backend, était fortement recommandée par l'organisme de formation. L'objectif était de se familiariser avec des outils professionnels et d'adopter une approche structurée du développement. De plus, le projet devait prendre en compte les contraintes budgétaires d'une petite structure comme un bar à jeux, privilégiant l'utilisation de technologies open source et d'une plateforme d'hébergement gratuite (Render).

Sur le plan technique, le projet devait être compatible avec les navigateurs web modernes (Chrome, Firefox, Safari, Edge) et s'adapter aux différentes tailles d'écran (ordinateur, tablette, smartphone).

Le respect des contraintes légales, et en particulier du Règlement Général sur la Protection des Données (RGPD), était un impératif. Le projet devait donc intégrer les principes de protection des données personnelles dès la conception (privacy by design) et garantir la sécurité des informations collectées.

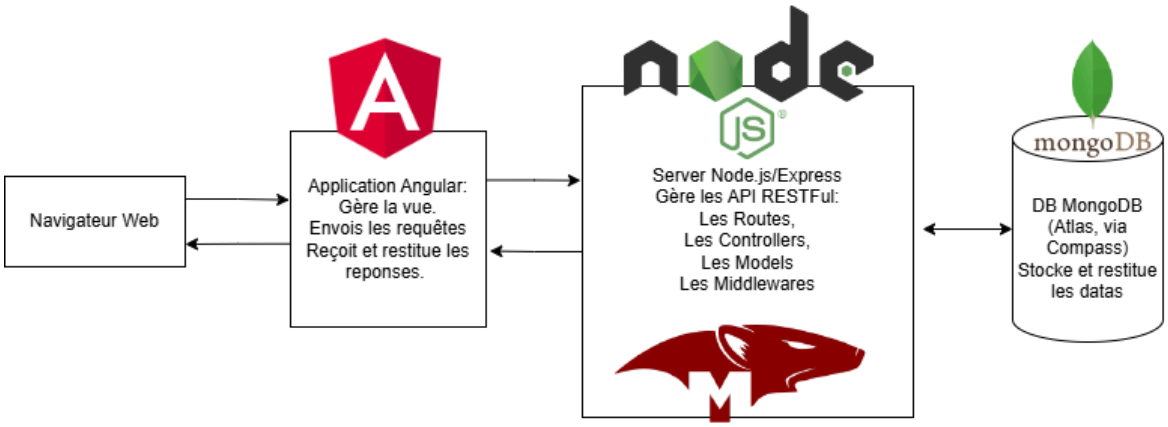
Les livrables attendus à l'issue du projet étaient les suivants :

- Une application web fonctionnelle, accessible en ligne via une URL publique fournie par la plateforme d'hébergement Render.
- Le code source complet et commenté de l'application (frontend Angular et backend Node.js/Express), hébergé sur un dépôt Git public (GitHub).
- Le présent dossier de présentation du projet, détaillant les différentes phases de conception et de réalisation.
- Une présentation orale du projet, incluant une démonstration de l'application.

Gestion du projet

1. Environnement humain et technique

Le projet TGMT a été mené individuellement, dans le cadre d'un apprentissage intensif des technologies de développement web. J'ai cependant pu bénéficier de l'encadrement et des conseils précieux de l'équipe encadrante de l'école et de mes pairs, ainsi que de la collaboration de testeurs de l'application.



L'environnement technique du projet est le suivant :

- **Frontend :** Le développement de l'interface utilisateur repose sur le framework Angular (18.2.1). Ce choix a été motivé par la richesse fonctionnelle d'Angular, sa structure modulaire, et sa capacité à gérer des applications web complexes de manière efficace. L'utilisation de TypeScript, un sur-ensemble de JavaScript, a permis d'améliorer la qualité et la maintenabilité du code.
- **Backend :** L'API RESTful a été construite avec Node.js (18.20.4) et le framework Express (4.19.2). La légèreté, la rapidité et la grande communauté de Node.js, ainsi que la simplicité d'Express, en ont fait des choix naturels pour ce projet.
- **Base de données :** La persistance des données est assurée par MongoDB (version 8.0.4) une base de données NoSQL, utilisée via la librairie Mongoose (8.6.0). La flexibilité de MongoDB, et sa capacité à gérer des données non structurées, ont été des atouts majeurs pour ce projet.

- **Authentification :** La sécurisation de l'accès aux ressources sensibles est réalisée grâce à l'implémentation de JSON Web Tokens (JWT).
- **Hachage des mots de passe :** La librairie bcrypt a été employée pour le hachage des mots de passe, garantissant une sécurité renforcée des données utilisateur.
- **Gestionnaire de paquets :** npm (10.9.1) a été utilisé pour la gestion des dépendances du projet.
- **Contrôle de version :** Git, associé à la plateforme GitHub, a permis de suivre l'évolution du code source, de faciliter le travail collaboratif (même en solo, pour l'historique), et d'assurer la sauvegarde du projet.
- **Hébergement :** L'application est déployée sur la plateforme Render, choisie pour sa simplicité d'utilisation et son plan gratuit adapté à un projet de démonstration.
- **IDE :** Le développement a été effectué avec l'éditeur de code VS Code, choisi pour sa richesse fonctionnelle et son intégration avec les outils de développement web.
- **Autres outils :** Postman (pour tester l'API), MongoDB Compass (pour visualiser la base de données).
- **Librairies tierces :** Outre les librairies principales déjà mentionnées (Angular, Node.js, Express, Mongoose, etc.), le projet s'appuie sur diverses librairies tierces pour des fonctionnalités spécifiques (gestion des formulaires, éditeur de texte riche, etc.).

La liste exhaustive de ces librairies, avec leurs versions et une brève description de leur rôle, est fournie en Annexe (2).

2. Objectifs qualité

La qualité du produit final a été une préoccupation constante tout au long du projet. Les objectifs suivants ont été fixés :

- **Accessibilité :** L'application a été conçue pour être "responsive", c'est-à-dire s'adapter automatiquement aux différentes tailles d'écran (ordinateur, tablette, smartphone). Cet objectif a été atteint grâce à l'utilisation des classes utilitaires de Bootstrap et de media queries CSS.

- **Sécurité :** La protection des données utilisateur et la prévention des attaques ont été traitées avec la plus grande attention. Les mots de passe sont hachés avec l'algorithme bcrypt avant d'être stockés en base de données. L'authentification des utilisateurs est gérée par des tokens JWT, et les routes sensibles sont protégées par des middlewares (côté serveur) et des guards (côté client). Les injections XSS sont évitées grâce à l'utilisation du DomSanitizer d'Angular.
- **Utilisabilité :** Une attention particulière a été portée à l'ergonomie de l'application. L'interface utilisateur est simple et intuitive, la navigation est fluide, et les informations sont présentées de manière claire. L'interface utilisateur a été conçue en s'inspirant des bonnes pratiques d'ergonomie et en utilisant des composants réutilisables (comme game-card) pour assurer une cohérence visuelle.

- **Performance :** L'application a été conçue pour être réactive. L'utilisation d'Angular, avec son système de détection des changements optimisé, et de Node.js/Express, réputé pour sa rapidité, contribue à cet objectif. Le tri des tables de jeu est effectué côté serveur (via Mongoose) pour minimiser la quantité de données transférées. entre le serveur et le client. Les images sont optimisées pour un chargement rapide. Les URLs des images sont stockées de manière relative pour assurer la portabilité.

- **Maintenabilité :** Le code a été structuré de manière modulaire (composants et services Angular, contrôleurs, modèles et routes Express). L'utilisation de TypeScript, avec son typage statique, facilite la compréhension et la maintenance du code. Des variables CSS ont été utilisées pour centraliser les styles et faciliter les modifications ultérieures. Le code est commenté.

Réalisations front-end

L'interface utilisateur de l'application TGMT a été entièrement développée avec le framework Angular (version 18.2.1). Ce choix, motivé par sa richesse fonctionnelle, sa structure modulaire favorisant la maintenabilité et l'évolutivité du code, et son utilisation de TypeScript (typage statique, meilleure détection des erreurs), s'inscrit dans une volonté d'adopter des technologies modernes et performantes. L'architecture modulaire d'Angular, basée sur des composants réutilisables et des services injectables, est, de plus, particulièrement adaptée au développement d'applications web complexes et évolutives, amenées à s'enrichir de nouvelles fonctionnalités.

Cette section a pour objectif de présenter concrètement les réalisations frontend de l'application, en mettant en évidence les compétences acquises en matière de développement avec Angular. Nous commencerons par présenter l'organisation générale du code source, en décrivant brièvement les composants, services, modules, le système de routing et les guards utilisés. Nous aborderons ensuite la conception de l'interface utilisateur : présentation des maquettes, description des techniques d'adaptation web et web mobile (responsive design), et présentation du plan du site. Nous détaillerons ensuite l'interface utilisateur elle-même, en suivant une approche structurée par profil utilisateur (visiteur non connecté, utilisateur connecté standard, administrateur), et en fournissant des captures d'écran annotées pour chaque page/fonctionnalité significative ainsi que sur des extraits de code, accompagnés d'explications détaillées, afin d'illustrer les points clés de l'implémentation et les bonnes pratiques de développement Angular mises en œuvre.

1. Organisation du code et présentation générale d'Angular

L'application TGMT est structurée selon les principes fondamentaux d'Angular, favorisant une organisation claire, modulaire et maintenable. Le code source est organisé en composants, services, modules, et modèles, chacun ayant un rôle spécifique.

a . Composants :

Les composants sont les éléments de base de l'interface utilisateur Angular. Chaque composant contrôle une partie de l'écran (un "morceau" de l'interface) et est composé de trois parties :

- un fichier TypeScript (.component.ts) contenant la logique,
- un fichier HTML (.component.html) définissant le template,
- un fichier CSS (.component.css) pour les styles spécifiques au composant.

Par exemple, le composant GameCardComponent (fichiers *game-card.component.ts*, *game-card.component.html*, *game-card.component.css*) est responsable de l'affichage d'une carte de jeu. Il reçoit les données du jeu en entrée et les affiche dans le Template HTML :

```
import { Component, OnInit } from '@angular/core';
import { GameService } from '../../services/game.service'; // Import the GameService to interact with game data.

// Component decorator to define metadata for the component.
@Component({
  selector: 'app-gamecard', // Selector for using the component in templates.
  templateUrl: './gamecard.component.html', // Path to the component's HTML template.
  styleUrls: ['./gamecard.component.css'] // Path to the component's CSS styles.
})
export class GamecardComponent implements OnInit {
  games: any[] = []; // Array to store the list of games.
  searchName: string = ''; // Variable to store the search term for game name.
  searchType: string = ''; // Variable to store the search term for game type.
  searchCategory: string = ''; // Variable to store the search term for game category.
  searchAgeMin: number | null = null; // Variable to store the search term for minimum age.
  searchDifficulty: string = ''; // Variable to store the search term for game difficulty.
  searchAuthor: string = ''; // Variable to store the search term for game author.
  searchPlayerMin: number | null = null; // Variable to store the search term for minimum number of players.
  searchPlayerMax: number | null = null; // Variable to store the search term for maximum number of players.
  searchPartytime: number | null = null; // Variable to store the search term for game duration.

  // Constructor to inject the GameService.
  constructor(private gameService: GameService) {}
```

```
<div class="row">
  <!-- Using *ngFor to iterate over the list of games -->
  <div class="col-md-3" *ngFor="let game of games">
    <!-- Square representing each game -->
    <div class="game-card">
      <!-- Clickable link to the game page -->
      <a [routerLink]="['/single-game', game._id]">
        <!-- Game image with link -->
        <img class="game-image" [src]="game.image" alt="Game Image">
      </a>

      <!-- name of the game -->
      <div class="game-title">{{ game.name }}</div>

      <!-- Game details in two columns -->
      <div class="game-details">
        <!-- First column -->
        <div class="column">
          <div><span class="orange-text">Type:</span> {{ game.type }}</div>
          <div><span class="orange-text">Catégorie:</span> {{ game.category }}</div>
        </div>

        <!-- second column -->
        <div class="column">
          <div><span class="orange-text">Âge minimum:</span> {{ game.ageMin }}</div>
          <div><span class="orange-text">Difficulté:</span> {{ game.difficulty }}</div>
          <div><span class="orange-text">Nombre de joueurs:</span> {{ game.playerMin }} - {{ game.playerMax }}</div>
          <div><span class="orange-text">Temps de jeu:</span> {{ game.partytime }}</div>
        </div>
      </div>
    </div>
  </div>
</div>
```

Cet extrait montre comment le composant *GameCardComponent* est structuré. Le fichier TypeScript *game-card.component.ts* importe les modules nécessaires, définit le composant avec le décorateur '@Component', déclare les propriétés du composant (dont la liste des jeux), et injecte le service *GameService.*

Le template HTML (game-card.component.html) utilise la directive *ngFor pour parcourir la liste des jeux et afficher une carte pour chaque jeu. La liaison de données ([src]="game.image", {{ game.name }}) permet d'afficher les informations du jeu. Le lien [routerLink] permet quant à lui de rediriger vers la page de détail du jeu correspondant.

Un autre exemple est le composant CreateTableComponent, qui gère le formulaire de création d'une table de jeu. Ce composant utilise un FormGroup pour gérer les données du formulaire et les validateurs Angular pour s'assurer de la validité des données saisies :

```
import { Component, OnInit } from '@angular/core';
import { FormBuilder, FormGroup, Validators, ValidatorFn, AbstractControl, ValidationErrors } from '@angular/forms';
import { GameService } from '../../services/game.service';
import { Router, ActivatedRoute } from '@angular/router'; // Imports ActivatedRoute
import { Game } from '../../models/game';
import { AuthService } from '../../services/auth.service';
import { DatePipe } from '@angular/common';

@Component({ ···
})
export class CreateTableComponent implements OnInit {
  createTableForm: FormGroup;
  games: Game[] = [];
  successMessage: string | null = null;
  errorMessage: string | null = null;
  userId: string | null = null;
  gameId: string | null = null;
  selectedGame: Game | undefined;

  constructor(
    private fb: FormBuilder,
    private gameService: GameService,
    private router: Router,
    private route: ActivatedRoute,
    private authService: AuthService,
    private datePipe: DatePipe
  ) {
    this.createTableForm = this.fb.group({
      game: ['', Validators.required],
      startTime: ['', Validators.required],
      duration: ['', [Validators.required, Validators.min(1)]],
      endTime: [{value: '', disabled: true}],
      creator: [''],
      maxParticipants: ['', [Validators.required]]
    });
  }
```

On voit la définition des différents champs du formulaire (game, startTime, duration, etc.) et l'utilisation de validateurs (par exemple, *Validators.required* pour rendre un champ obligatoire, *Validators.min(1)* pour imposer une valeur minimale). Ce formulaire est utilisé dans le template HTML du composant pour permettre à l'utilisateur de créer une nouvelle table de jeu.

La liste complète des composants est disponible en annexe 3.

b. Services :

Les services Angular sont des classes qui encapsulent la logique métier de l'application et les interactions avec le backend. Ils sont injectables dans les composants, ce qui permet de partager du code et des données entre plusieurs composants, et de séparer la logique de présentation de la logique métier.

Par exemple , le service GameService est responsable de la communication avec l'API backend pour la gestion des jeux. Il fournit des méthodes pour récupérer la liste des jeux, rechercher un jeu, créer un nouveau jeu, etc. Ces méthodes utilisent le service HttpClient d'Angular pour effectuer des requêtes HTTP :

```
nt > src > app > services > TS game.service.ts > GameService
import { Injectable } from '@angular/core';
import { HttpClient, HttpHeaders, HttpParams } from '@angular/common/http';
import { Observable } from 'rxjs';
import { Game } from '../models/game';
import { GameResponse } from '../models/game-response';
import { environment } from '@environments/environment';

@Injectable({
  providedIn: 'root'
})
export class GameService {
  private apiUrl = environment.apiUrl;
  private baseUrl = environment.apiUrl + '/api/games';
  private tableUrl = environment.apiUrl + '/api/tables';

  constructor(private http: HttpClient) { }

  getGames(): Observable<Game[]> {
    return this.http.get<Game[]>(`${this.baseUrl}`);
  }
```

La méthode *getGames()* retourne un Observable*<Game[]>*, ce qui signifie qu'elle renvoie un flux de données asynchrone (ici une liste de jeux). Le composant qui appelle cette méthode devra s'abonner à cet observable pour récupérer les données.

Le service AuthService, qui gère l'authentification des utilisateurs, fournit des méthodes pour se connecter, se déconnecter, récupérer le rôle de l'utilisateur, etc. Il utilise des BehaviorSubject pour gérer l'état de connexion de l'utilisateur et le partager avec les autres composants :

```
import { Injectable } from '@angular/core';
import { HttpClient, HttpErrorResponse, HttpHeaders } from '@angular/common/http';
import { BehaviorSubject, throwError, Observable } from 'rxjs';
import { catchError, tap } from 'rxjs/operators';
import { Router } from '@angular/router';
import { environment } from '@environments/environment';

@Injectable({
  providedIn: 'root'
})
export class AuthService {
  private apiUrl = environment.apiUrl + '/users';
  public roleSubject = new BehaviorSubject<string | null>(null);
  role$ = this.roleSubject.asObservable();

  public userIdSubject = new BehaviorSubject<string | null>(null);

  errorMessage: string = '';
  private tokenKey = 'token';

  constructor(private http: HttpClient, private router: Router) {…
  }
```

Les BehaviorSubject permettent de stocker une valeur (ici, le rôle et l'ID de l'utilisateur) et de la notifier à tous les composants qui y sont abonnés. La propriété *role$* est un observable qui émet la valeur courante du rôle, et les composants peuvent s'y abonner pour être informés des changements.

c. Modules :

L'application est organisée en un module principal, AppModule, qui importe les modules nécessaires au fonctionnement de l'application (par exemple, HttpClientModule pour les requêtes HTTP, FormsModule et ReactiveFormsModule pour la gestion des formulaires, RouterModule pour le routing, etc.).

```
import { NgModule, CUSTOM_ELEMENTS_SCHEMA } from '@angular/core';
import { BrowserModule } from '@angular/platform-browser';
import { HttpClientModule, HTTP_INTERCEPTORS, provideHttpClient, withFetch }
import { FormsModule, ReactiveFormsModule } from '@angular/forms';
```

```
@NgModule({
  declarations: [ ···
  ],
  imports: [
    BrowserModule,
    AppRoutingModule,
    HttpClientModule,
    FormsModule,
    ReactiveFormsModule,
    QuillModule.forRoot()
  ],
  providers: [
    { provide: HTTP_INTERCEPTORS, useClass: AuthInterceptor, multi: true },
    provideHttpClient(withFetch()) // Add withFetch for se fetch API with SS
  ],
  bootstrap: [AppComponent],
  schemas: [CUSTOM_ELEMENTS_SCHEMA]
})
export class AppModule { }
```

La section déclarations liste les composants utilisés dans l'application, et la section imports liste les modules importés.

d. Modèles :

Le rôle des modèles est de définir la structure des données utilisées dans l'application. Par exemple, l'interface Game définit les propriétés d'un objet jeu. Ils sont utilisés pour typer les données et faciliter la maintenance du code.

```
export interface Game {
  _id: string;
  name: string;
  type: string;
  category: string;
  ageMin: number;
  difficulty: string;
  author: string;
  playerMin: number;
  playerMax: number;
  description: string;
  image: string;
  partytime: number;
  rules: string;
  resourcesLink: string[];
  createdAt: Date;
  tables: any[];
  isFavorite: boolean;
}
```

Cela permet au compilateur TypeScript de vérifier que les données sont utilisées correctement, et d'éviter les erreurs. Par exemple, si on essaie d'accéder à une propriété qui n'existe pas sur un objet Game, le compilateur signalera une erreur.

e. Routing :

L'application utilise le routeur Angular pour gérer la navigation entre les différentes pages. Le fichier *app-routing.module.ts* définit les routes de l'application, en associant chaque URL à un composant.

```
import { NgModule } from '@angular/core';
import { RouterModule, Routes } from '@angular/router';
import { SignupComponent } from './components/auth/signup/signup.component';
import { SigninComponent } from './components/auth/signin/signin.component';
import { GamecardComponent } from './components/gamecard/gamecard.component';
import { SingleGameComponent } from './components/gamecard/single-game/single-game.component';
import { DashboardComponent } from './components/dashboard/dashboard.component';
import { UserInfoComponent } from './components/user-info/user-info.component';
// ... autres imports ...

const routes: Routes = [
  { path: 'sign-up', component: SignupComponent },
  { path: 'sign-in', component: SigninComponent },
  { path: 'home', component: HomeComponent },        // AJOUTE
  { path: 'contact', component: ContactComponent },
  { path: 'gamecard', component: GamecardComponent },
  { path: 'single-game/:id', component: SingleGameComponent },
  { path: 'games/:id/update', component: UpdateGameComponent, canActivate: [AuthGuard, AdminGuard], data: { expectedRole: 'admin' }},
  { path: 'dashboard', component: DashboardComponent, canActivate: [AuthGuard] },
  { path: 'mes-informations', component: UserInfoComponent, canActivate: [AuthGuard] },
  // ... autres routes ...
  // Default Redirection
  { path: '', redirectTo: '/home', pathMatch: 'full' },
  { path: 'admin/opening-hours', component: OpeningHoursComponent, canActivate: [AdminGuard] },
];
@NgModule({
  imports: [RouterModule.forRoot(routes)],
  exports: [RouterModule]
})
export class AppRoutingModule { }
```

Chaque route associe un chemin (path) à un composant (component).

Par exemple, la route *{ path: 'tables', component: TableListComponent, canActivate: [AuthGuard] }* indique que l'URL */tables* doit afficher le composant *TableListComponent*, et que cette route est protégée par le guard *AuthGuard.*

La route *{ path: 'single-game/:id', component: SingleGameComponent }* montre comment passer un paramètre (ici, l'ID du jeu) dans l'URL.

La route *{ path: '', redirectTo: '/gamecard', pathMatch: 'full' }* définit la redirection par défaut (vers */gamecard* dans cet exemple).

La propriété *canActivate: [AuthGuard]* indique que la route est protégée par un guard (dans ce cas, *AuthGuard*).

f. Guards :

Les guards Angular sont des services spéciaux qui permettent de contrôler l'accès aux routes de l'application. Ils implémentent l'interface CanActivate et définissent une méthode canActivate qui retourne un booléen (ou un Observable/Promise de booléen) indiquant si la navigation vers une route est autorisée ou non. Les guards sont utilisés pour implémenter des logiques d'authentification et d'autorisation.

L'application TGMT utilise deux guards :

- **AuthGuard :** Ce guard vérifie si l'utilisateur est connecté. Il est utilisé pour protéger les routes qui nécessitent une authentification.

```
import { Injectable } from '@angular/core';
import { CanActivate, ActivatedRouteSnapshot, RouterStateSnapshot, Router } from '@angular/router';
import { AuthService } from '../services/auth.service';
import { Observable, of } from 'rxjs';
import { map, catchError } from 'rxjs/operators';

@Injectable({
  providedIn: 'root'
})
export class AuthGuard implements CanActivate {

  constructor(private authService: AuthService, private router: Router) {}

  canActivate(
    next: ActivatedRouteSnapshot,
    state: RouterStateSnapshot
  ): Observable<boolean> {
    return this.authService.getRole().pipe(
      map(role => {
        const expectedRole = next.data['expectedRole']; // Gets the expected role

        if (!role) {
          // If the user is not logged in, redirect to the login page
          this.router.navigate(['/sign-in'], { queryParams: { returnUrl: state.url } });
          return false;
        }

        if (expectedRole && role !== expectedRole) {
          // If a role is expected and the user doesn't have it, redirect
          // (you can customize the redirection here)
          this.router.navigate(['/']); // Redirect to home, for example
          return false;
        }
        // If the user is logged in and has the expected role (or if no role is expected), allow access
        return true;
      }),
      catchError(() => {
        // In case of an error while retrieving the role (for example, token expired)
        this.router.navigate(['/sign-in'], { queryParams: { returnUrl: state.url } });
        return of(false); // Block access and force logout/redirection
      })
    );
  }
```

Il injecte le service *AuthService* et le service *Router*.

La méthode *canActivate* retourne un observable de booléen. Elle utilise la méthode *getUserId()* du *AuthService* pour récupérer l'ID de l'utilisateur connecté (ou *null* si l'utilisateur n'est pas connecté).

Si un ID utilisateur est présent, la méthode retourne *true* (l'accès est autorisé). Sinon, elle redirige l'utilisateur vers la page de connexion (en passant l'URL de retour en paramètre), et retourne *false* (l'accès est refusé).

- **AdminGuard :** Ce guard vérifie si l'utilisateur est connecté *et* s'il a le rôle 'admin'. Il est utilisé pour protéger les routes qui nécessitent des privilèges d'administrateur.

```
import { Injectable } from '@angular/core';
import { CanActivate, Router } from '@angular/router';
import { AuthService } from '../services/auth.service';
import { Observable } from 'rxjs';
import { map } from 'rxjs/operators';

@Injectable({
  providedIn: 'root'
})
export class AdminGuard implements CanActivate {

  constructor(private authService: AuthService, private router: Router) {}

  canActivate(): Observable<boolean> {
    return this.authService.getRole().pipe(
      map(role => {
        if (role === 'admin') {
          return true; // Allow access to the admin/dashboard route
        } else {
          this.router.navigate(['/sign-in']); // Redirect to the login page if the user is not an admin
          return false;
        }
      })
    );
  }
}
```

Comme le *AuthGuard,* il injecte le service *AuthService* et le service Router et la méthode *canActivate* retourne un observable de booléen. Elle utilise la méthode *getRole()* du *AuthService* pour récupérer le rôle de l'utilisateur connecté.

Cette fois, si le rôle est *admin*, la méthode retourne *true* (l'accès est autorisé). Sinon, elle redirige l'utilisateur vers la page de connexion et retourne *false* (l'accès est refusé).

Ces guards sont utilisés dans le fichier de routing *app-routing.module.ts* pour protéger les routes. Par exemple :

```
{
  path: 'admin/dashboard',
  component: DashboardComponent,
  canActivate: [AuthGuard, AdminGuard],
  data: { expectedRole: 'admin' },

},
  { path: 'create-game', component: CreateGameComponent, canActivate: [AdminGuard]},
  { path: 'user-management', component: UserManagementComponent, canActivate: [AdminGuard]},
{ path: 'dashboard/favorites', component: FavoritesComponent, canActivate: [AuthGuard]},

// Default Redirection
{ path: '', redirectTo: '/home', pathMatch: 'full' },
{ path: 'admin/opening-hours', component: OpeningHoursComponent, canActivate: [AdminGuard] },
```

g. Directives et pipes :

Angular propose des mécanismes pour manipuler le DOM et transformer les données à afficher : les directives et les pipes.

Les directives étendent le comportement des éléments HTML ou modifient la structure du DOM. Il existe trois types de directives :

- **Composants :** Déjà abordés, ce sont les éléments de base de l'interface.
- **Directives structurelles :** Elles modifient la structure du DOM par ajout/suppression d'éléments. Elles sont préfixées par un astérisque `*`.
- **Directives d'attribut :** Elles modifient l'apparence ou le comportement d'un élément.

L'application TGMT utilise abondamment les directives structurelles fournies par Angular :

- ***ngIf*** : Affichage conditionnel d'un élément. Exemple (extrait de *single-game.component.html*) : Le bouton "Créer une table pour ce jeu" n'est affiché que si la propriété *isLoggedIn* du composant est vraie, donc si l'utilisateur est connecté.

```
<button class="btn btn-primary btn-custom-gradient w-100 mt-3" *ngIf="isLoggedIn" (click)="openCreateTableForm()">Créer une table pour ce jeu</button>
```

- ***ngFor*** **:** Répétition d'un élément HTML pour chaque élément d'un tableau.

```
<button class="btn btn-primary btn-custom-gradient w-100 mt-3" *ngIf="isLoggedIn" (click)="openCreateTableForm()">Créer une table pour ce jeu</button>
```

```
<div *ngFor="let table of tables" class="col-md-3 mb-3">
```

La directive **ngFor* est utilisée pour parcourir des tableaux et afficher des éléments de manière répétitive. Dans le premier exemple, une *div* avec la classe *game-card* est créée pour chaque jeu dans le tableau *games*. Dans le second, une *div* est crée pour chaque *table*.

L'application utilise également la directive d'attribut **[ngClass]** pour appliquer des classes CSS de manière conditionnelle.

```
<strong>Statut:</strong>
  <span [ngClass]="{'text-success': table.open, 'text-danger': !table.open}">
  {{ table.open ? 'Ouverte' : 'Fermée' }}</span><br>
```

La classe *text-success* (Bootstrap) est appliquée si *table.open* est vrai, et la classe *text-danger* est appliqué si *table.open* est faux.

Les pipes transforment les données avant de les afficher. Angular fournit des pipes intégrés, et il est possible d'en créer des personnalisés.

TGMT utilise le pipe date pour formater les dates et heures.

```
<h5 class="card-title">{{ table.startTime | date:'dd/MM HH:mm' }}</h5>
```

Le pipe *date* est appliqué à la propriété *table.startTime*. L'argument *dd/MM HH:mm* spécifie le format d'affichage souhaité (jour/mois, heure:minutes).

2. Organisation de l'interface

a. Maquettes :

La conception de l'interface utilisateur de l'application TGMT a été précédée d'une phase de maquettage, afin de visualiser l'organisation des pages et des éléments d'interface, et de valider les choix ergonomiques. Celles-ci sont disponible en annexe (3).

b. Adaptation Web et Web Mobile (Responsive Design)

L'application TGMT a été conçue pour être entièrement responsive, c'est-à-dire qu'elle s'adapte à toutes les tailles d'écran, des ordinateurs de bureau aux smartphones, en passant par les tablettes. Cette adaptabilité est essentielle pour offrir une expérience utilisateur optimale, quel que soit le terminal utilisé.

Pour garantir cette responsivité, plusieurs techniques ont été mises en œuvre :

- **Utilisation du framework Bootstrap :** L'application utilise le framework CSS Bootstrap, et en particulier son système de grille, pour organiser les éléments de l'interface de manière flexible. Les classes Bootstrap, telles que *col-md-4*, *col-sm-6*, etc., permettent de définir le comportement des éléments en fonction de la largeur de l'écran. Par exemple, dans la liste des jeux (game-card.component.html) et la liste des tables (table-list.component.html), les cartes de jeux/tables sont affichées sur trois colonnes sur les grands écrans, et passent automatiquement à deux colonnes, puis une seule, sur les écrans plus petits. En plus des classes de grille, des classes utilitaires Bootstrap ont été utilisées pour gérer les espacements, l'alignement du texte, les boutons, etc.

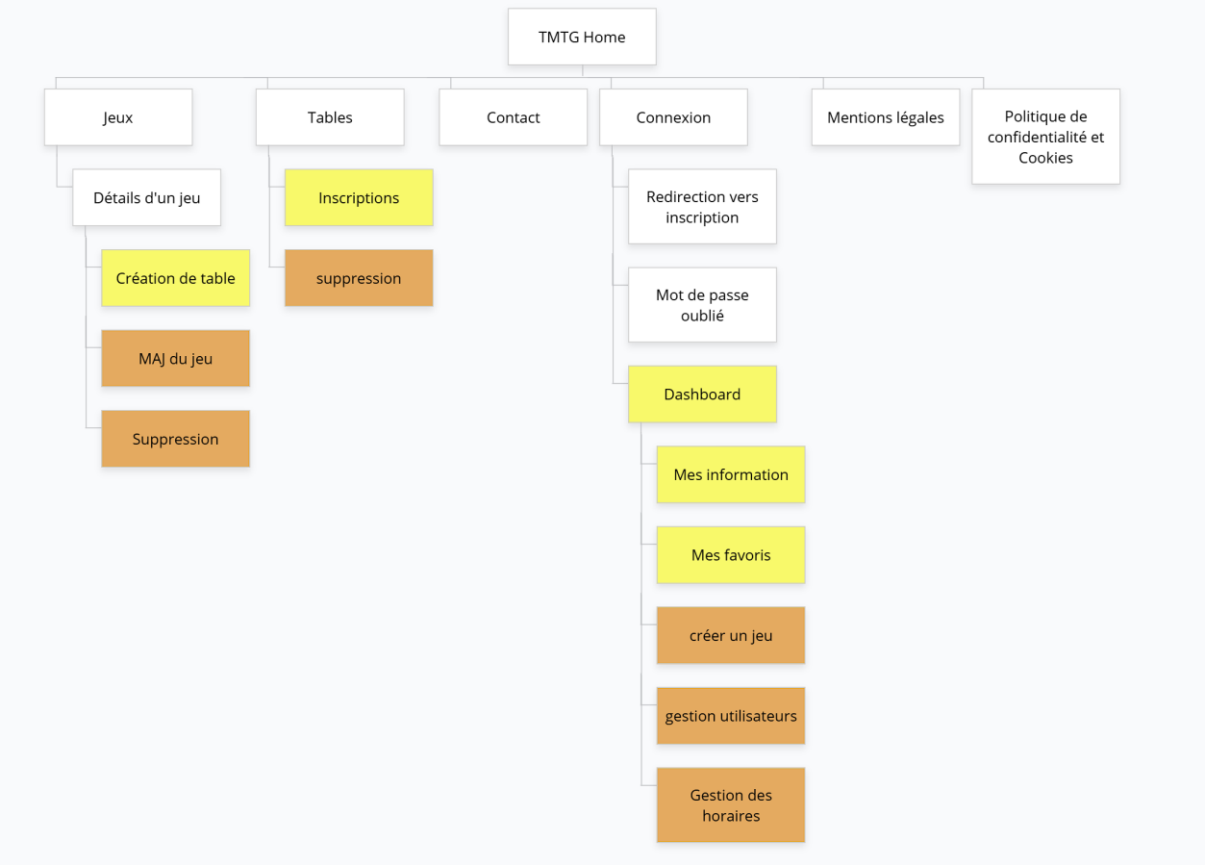
- **Utilisation de Media Queries CSS :** En complément de Bootstrap, des media queries CSS ont été utilisées pour ajuster certains styles en fonction de la taille de l'écran. Cela permet d'affiner le comportement de l'application et d'optimiser l'expérience utilisateur sur des points précis. Un exemple particulièrement significatif est la transformation du menu de navigation principal en menu "hamburger" sur les petits écrans.

```css
/* Styles for the responsive menu */
@media (max-width: 664px) {
  .nav-content .nav {
    display: none; /* Hide the menu by default on small screens */
    border-radius: var(--spacing-sm);
    position: absolute; /* Absolute positioning */
    top: 100%; /* Position below the header (100% of the header's height) */
    right: 0; /* Align to the right */
    background-color: var(--light-gray-color);
    z-index: 10;
    padding: var(--spacing-sm);
    border: 1px solid var(--border-color); /* Added a border */
    min-width: 150px;  /* Minimum width */

  }

  .nav-content {
    position: relative; /* Added position: relative; */
  }

  .nav-content .nav.show {
    display: block; /* Display the menu when it has the 'show' class */
    position: absolute;
    top: 100%; /* Positions the menu just below the header */
    right: 0; /* Keeps the right alignment */
    width: auto; /* Width adapts to content */
    min-width: 150px; /* Minimum width to avoid a menu that's too narrow */
    background-color: var(--light-gray-color);
    z-index: 10; /* Ensure it's above the rest of the content */
    padding: var(--spacing-sm); /* Added padding for the menu */
    border: 1px solid var(--border-color); /* Added a border */
    border-radius: var(--spacing-sm); /* Rounded corners */
  }

  .nav-content .nav ul {
    display: flex;
    flex-direction: column; /* Display links vertically */
    align-items: flex-start; /* Align links to the left */
    padding: 0;
    margin: 0;
  }

  .nav-content .nav ul li {
    width: 100%; /* List items take up the full width */
    margin-bottom: var(--spacing-sm); /* Margin between links */
  }

  .nav-content .nav ul li a {
      display: block; /* Transform links into blocks for easier clicking */
      padding: var(--spacing-xs) var(--spacing-sm); /* Padding around links */
  }

  .hamburger-menu {
    display: block; /* Display the hamburger button on small screens */
  }

  .social-icons {
    display: none;
  }
  .material-symbols-outlined.rotated {
    transform: rotate(90deg); /* 90-degree rotation */
  }
}
```

Lorsque la largeur de l'écran est inférieure au breakpoint (664px), le menu principal est masqué, et le bouton hamburger est affiché. Lorsque l'utilisateur clique sur le bouton hamburger, la classe show est ajoutée dynamiquement au menu, ce qui le rend visible. Le menu est positionné de manière absolue, juste en dessous du header et aligné à droite. La propriété position: relative est appliqué à la div nav-content pour permettre un positionnement absolute pour le menu. Enfin, la classe rotated permet de gerer la rotation de l'icône du menu pour une indication visuelle.

c. Site map

Le plan du site (sitemap) de l'application TGMT est présenté ci-dessous. Il illustre l'organisation des différentes pages et leur accessibilité en fonction du statut de l'utilisateur.



- **Fond blanc :** Ces pages sont accessibles à tous les visiteurs, qu'ils soient connectés ou non.
- **Fond jaune :** Ces pages nécessitent une authentification préalable. Elles sont accessibles à tous les utilisateurs disposant d'un compte (joueurs et administrateurs).
- **Fond orange :** Ces pages sont réservées aux utilisateurs disposant du rôle "administrateur".

3. Interface utilisateur – présentation générale et exemples.

a. Éléments communs et pages statiques

Le header est composé du logo de l'application (également lien de retour à la homepage) et du menu de navigation (étalé ou sous forme de menu déroulant selon la taille de l'écran), ainsi que d'un ensemble de liens vers les réseaux sociaux liés à l'application. Les boutons du menu changent selon le statuts de l'utilisateur, défini par le AuthGuard.

La page d'accueil est conçue pour présenter clairement l'objectif de la plateforme et inciter les visiteurs à découvrir les fonctionnalités principales de l'application , regroupées par cartes. Chaque carte contient un titre, un court paragraphe descriptif et une captures d'écran.

```
import { Component } from '@angular/core';

@Component({
  selector: 'app-home',
  templateUrl: './home.component.html',
  styleUrl: './home.component.css'
})
export class HomeComponent {

}
```

Le code source du composant HomeComponent est minimaliste, car il n'y a pas de logique métier spécifique à cette page. Il se contente d'afficher le template HTML, et de relier la feuille de style correspondante.

Le footer, présent sur toutes les pages de l'application, est géré par le composant FooterComponent. Il affiche le nom de l'application, un copyright, et des liens vers les pages "Mentions légales" et "Politique de cookies".

La page de contact est prévue pour afficher les coordonnées du bar à jeux (adresse, téléphone, email) et proposer un formulaire de contact simple. Ce formulaire est actuellement statique (les données saisies ne sont pas traitées puisqu'il s'agit d'une page de démo).

De la même façon, la page des mentions légales (non présenté ici) est un template prévu pour afficher les informations liées à la structure adoptant l'application.

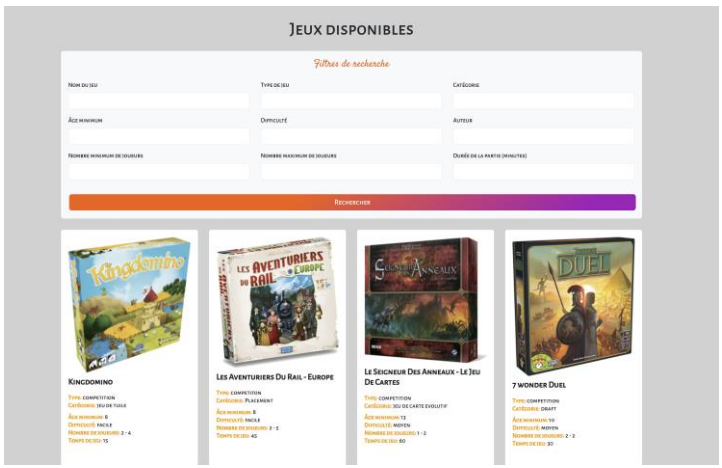
La pages de politique des cookies affichent un contenu textuel statique, expliquant l'utilisation des cookies faites par l'application et ls démarches permettant à l'utilisateur de s'en débarrasser.

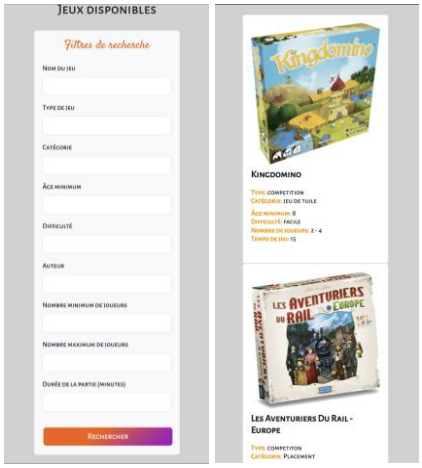
```
<footer>
  <div class="footer-container">
    <div class="footer-top">
      <ul>
        <li><a routerLink="/home">www.tgmt.fr</a></li>
        <li><a routerLink="/mentions-legales">Mentions légales</a></li>
        <li><a routerLink="/politique-cookies">Politique des Cookies</a></li>
      </ul>
    </div>

    <div class="footer-bot">
        <div class="container">
          <p>&copy; 2025 Tabletop Games Meeting Tool. Tous droits réservés.</p>
        </div>
    </div>
  </div>
</footer>
```

Cet extrait montre l'utilisation de la directive routerLink pour créer des liens internes vers les pages de mentions légales et de politique de cookies. Le reste du contenu du footer est statique.

b. Liste des jeux (game-card.component)

La page de liste des jeux affiche les jeux disponibles sous forme de cartes. Elle permet de rechercher un jeu par nom et de filtrer les jeux par différentes caractéristiques. Chaque carte de jeu contient un lien vers la page de détails du jeu.






```
<h4 class="text-center mb-4">Filtres de recherche</h4>
<form (submit)="onSearch()">
  <div class="row">
    <div class="col-md-4 mb-3">
      <label for="name" class="form-label">Nom du jeu</label>
      <input type="text" id="name" [(ngModel)]="searchName" name="name" class="form-control">
    </div>
    <div class="col-md-4 mb-3">
      <label for="type" class="form-label">Type de jeu</label>
      <input type="text" id="type" [(ngModel)]="searchType" name="type" class="form-control">
    </div>
```

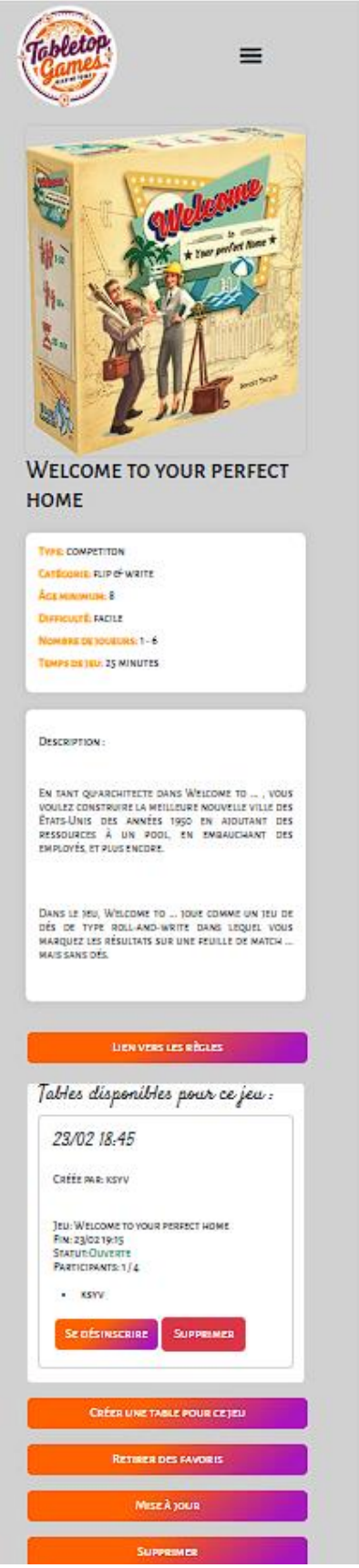
Cet extrait illustre l'utilisation de la liaison de données bidirectionnelle ([(ngModel)]) d'Angular.

Chaque champ de saisie (<input>) est lié à une propriété du composant (searchName, searchType, etc.) grâce à [(ngModel)]. Cela signifie que lorsque l'utilisateur tape du texte dans le champ, la valeur de la propriété correspondante du composant est *automatiquement* mise à jour. Inversement, si la valeur de la propriété du composant est modifiée par le code TypeScript, le champ de saisie affichera automatiquement la nouvelle valeur.

```
// Performs a search based on the entered criteria.
onSearch(): void {
  const searchCriteria = {
    name: this.searchName,
    type: this.searchType,
    category: this.searchCategory,
    ageMin: this.searchAgeMin,
    difficulty: this.searchDifficulty,
    author: this.searchAuthor,
    playerMin: this.searchPlayerMin,
    playerMax: this.searchPlayerMax,
    partytime: this.searchPartytime
  };

  this.gameService.searchGames(searchCriteria).subscribe(
    (data: any) => {
      this.games = data.result; // Updates the 'games' array with the search results.
    },
    (error: any) => {
      console.error('Error searching games:', error); // Logs an error message if the search fails.
    }
  );
}
```

Cette méthode, à la soumission du formulaire de recherche, construit un objet searchCriteria contenant les critères de recherche saisis par l'utilisateur. Elle appelle ensuite la méthode searchGames du service GameService pour effectuer la recherche.

c. Page d'un jeu (single-game.component)



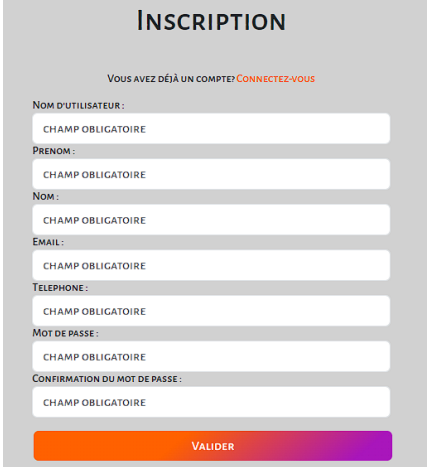
Cette page présente de manière détaillée les informations d'un jeu sélectionné. La section supérieure affiche les informations générales du jeu (nom, type, catégorie, âge minimum, difficulté, nombre de joueurs, temps de jeu, et image). En dessous, une description plus complète est fournie, suivie d'un lien externe vers les règles du jeu. La section centrale liste les tables de jeu disponibles pour ce jeu, avec, pour chaque table, les informations pertinentes (date et heure de début, créateur, statut, participants, et boutons d'action). Si l'utilisateur est connecté, des boutons lui permettent de s'inscrire ou de se désinscrire d'une table. S'il est administrateur, des boutons supplémentaires lui permettent de mettre à jour ou de supprimer le jeu, ou encore de supprimer une table. Enfin, un utilisateur connecté, quel que soit son rôle, peut ajouter le jeu à ses favoris, ou l'en retirer.

```
ngOnInit(): void {
  const gameId = this.route.snapshot.paramMap.get('id');

  if (gameId) {
    this.loadGameDetails(gameId);
    this.loadTables(gameId);
  } else {
    console.error('No game ID provided');
  }
```

```
loadGameDetails(gameId: string): void {
    this.gameService.getGameById(gameId).subscribe(
        response => {
            this.game = response.result;
            if (this.userId) {
                this.checkFavoriteStatus();
            }
        },
        error => {
            console.error('Error fetching game details:', error);
        }
    );
}
```

Le composant récupère l'ID du jeu à partir des paramètres de la route *ActivatedRoute* et utilise le *GameService* pour récupérer les détails du jeu depuis le backend. L'utilisation de subscribe montre la gestion des appels asynchrones avec les observables.

d. Connexion et inscription (sign-in, sign-up et reset-password.component)



La page de connexion présente un formulaire simple demandant à l'utilisateur de saisir son adresse e-mail et son mot de passe. Des liens permettent également à l'utilisateur de demander une réinitialisation de son mot de passe en cas d'oubli, ou de créer un compte s'il n'en possède pas encore.

La page de réinitialisation de mot de passe permet à un utilisateur ayant oublié son mot de passe de définir un nouveau mot de passe, après avoir suivi la procédure de récupération de mot de passe.

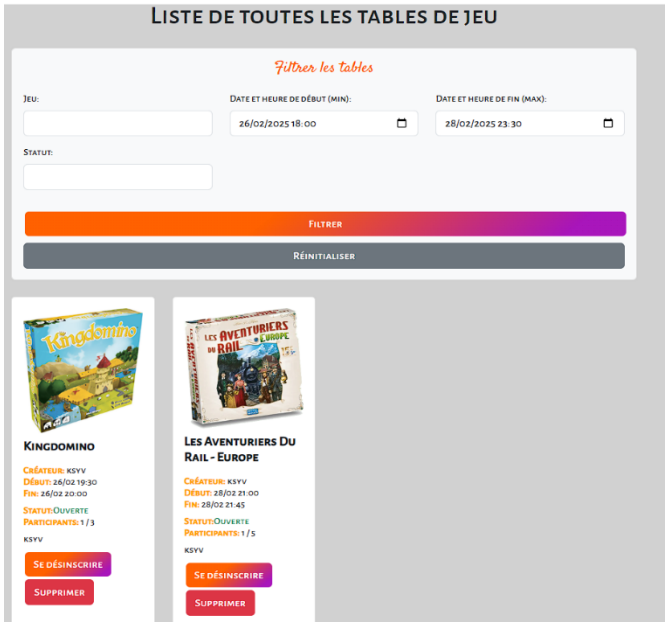
La page d'inscription permet à un nouvel utilisateur de créer un compte en fournissant les informations requises (nom d'utilisateur, prénom, nom, email, téléphone, mot de passe et confirmation du mot de passe).

```
onSubmit(form: NgForm) {
  this.errorForm = [];
  if (form.invalid) {
    return;
  }
  this.authService.login(this.signInData.email,
    this.signInData.password)
    .subscribe(
      (response: any) => {
        this.errorForm = [];
          // Redirection en fonction du rôle
      },
      (error: any) => {
        console.error('Login error:', error);
        this.errorForm.push({ general: 'Erreur de connexion. ' });
      }
    );
}
```

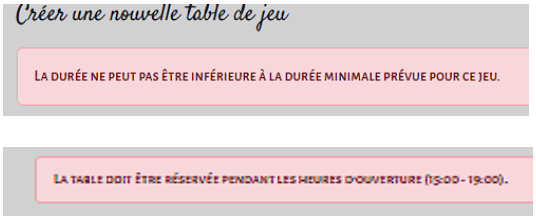
Cette méthode `onSubmit` du composant `SigninComponent` est appelée lorsque l'utilisateur soumet le formulaire de connexion. La méthode commence par réinitialiser le tableau d'erreurs (*this.errorForm*). Elle vérifie ensuite si le formulaire est valide (*form.invalid*).

Si le formulaire est valide, la méthode appelle le service AuthService (*this.authService.login*) pour authentifier l'utilisateur. La méthode login du AuthService envoie une requête POST au backend avec l'email et le mot de passe. Le composant s'abonne à la réponse du service (qui est un Observable). En cas de succès, l'utilisateur est redirigé. En cas d'erreur, un message d'erreur est ajouté au tableau *errorForm*. Une logique similaire est utilisée dans le composant SignupComponent pour gérer la soumission du formulaire d'inscription.

e. Liste et création de tables (pour utilisateur connecté)




```
this.createTableForm = this.fb.group({
  game: ['', Validators.required],
  startTime: ['', Validators.required],
  duration: ['', [Validators.required, Validators.min(1)]],
  endTime: [{value: '', disabled: true}],
  creator: [''],
  maxParticipants: ['', [Validators.required]]
});
```



La page de liste des table fonctionne de manière similaire à la page de liste des jeux, en ce qui concerne l'affichage en cartes, le formulaire de filtres et la structure générale.

La page de création de table, accessible via le bouton "Créer une table pour ce jeu" sur la page de détails d'un jeu (pour les utilisateurs connectés), permet de créer une nouvelle table de jeu associée à un jeu spécifique. Si un champs n'est pas renseigné ou une contraintes n'est pas respectée (nombre de joueurs min ou max, horaires d'ouverture de l'enseigne, jeu déjà réservé sur le même créneau), le message d'erreur associé s'affiche. Les contraintes sont comparés avec les valeurs connues pour chaque jeu par le back-end.

```
onSubmit() {
  this.successMessage = null;
  this.errorMessage = null;
  if (this.createTableForm.invalid) {
    return;
  }
  const formData = this.createTableForm.getRawValue();
  formData.startTime = new Date(formData.startTime);
  formData.endTime = new Date(formData.endTime);
  this.gameService.createGameTable(formData).subscribe({
    next: (response) => {
      console.log('Table créée avec succès:', response);
      this.successMessage = 'Table de jeu créée avec succès !';
      if (this.gameId) {
        this.router.navigate(['/single-game', this.gameId]);
      } else {
        console.error('gameId is not available for redirection.');
      }
```

```
      this.createTableForm.reset();
    },
    error: (error) => {
      console.error('Erreur lors de la création de la table:', error);
      if (error.error && error.error.error) {
        this.errorMessage = error.error.error;
      }
    }
  });
}
```

f. Le dashboard

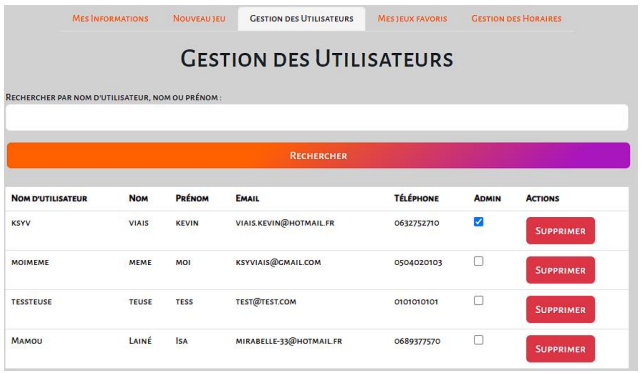
Le dashboard est le point d'entrée principal des utilisateurs après la connexion. Il affiche des liens vers les différentes sections de l'application accessibles à l'utilisateur. L'affichage du dashboard diffère selon le rôle de l'utilisateur (standard ou administrateur). Les utilisateurs standards ont accès à leurs informations et à la liste de leurs jeux favoris.

```
import { Component, OnInit } from '@angular/core';
import { AuthService } from '../../services/auth.service';

@Component({
  selector: 'app-dashboard',
  templateUrl: './dashboard.component.html',
  styleUrls: ['./dashboard.component.css']
})
export class DashboardComponent implements OnInit {
  isAdmin = false;
  selectedTab: string = 'mes-informations';

  constructor(private authService: AuthService) { }

  ngOnInit(): void {
    this.authService.getRole().subscribe(role => {
      this.isAdmin = (role === 'admin');
    });
    console.log("Initial selectedTab:", this.selectedTab);
  }
  selectManageHours() {
    this.selectedTab = 'manage-hours';
    console.log("selectedTab changed to:", this.selectedTab);
  }
}
```

Après les imports et la déclaration du component, on initialise les variable *isAdmin* à *false* et *selectedTab* à l'onglet *mes-informations*. Puis on injecte le service *AuthService* qui permet de savoir si un utilisateur est connecté, et son rôle. La fonction *ngOnInit* permet d'initialiser le composant, et de récupérer le rôle de l'utilisateur. La méthode *selectManageHours* permet de changer l'onglet sélectionné.

L'utilisateur standard aura accès aux onglets *Mes informations* et *Mes jeux favoris*. L'administrateur aura en plus *Nouveau jeu*, *Gestion des utilisateurs* et *Gestion des horaires*.

Mes informations permet de vérifier et de mettre à jours les information utilisateur (préremplies ave les informations existantes), y compris le mot de passe, ou de supprimer son compte. Dans ce cas, toute les informations liées à l'utilisateur ainsi que ses inscriptions éventuelles sur des tables sont supprimées.



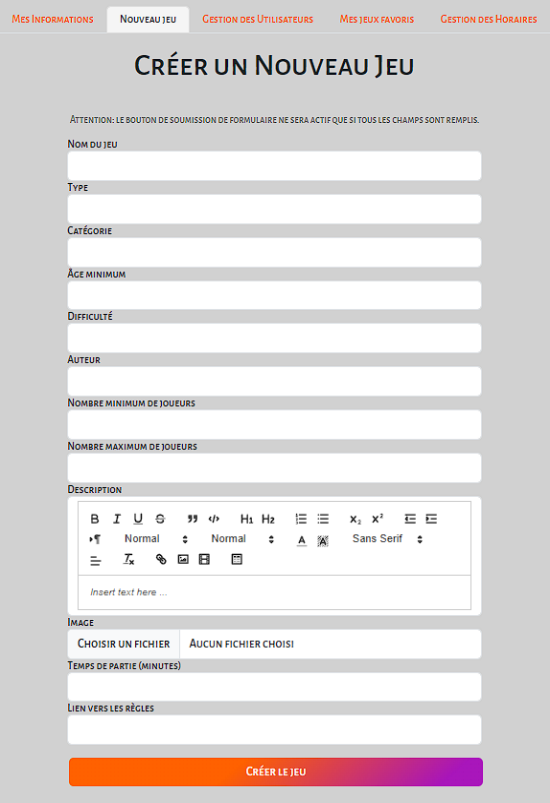
La gestion des utilisateurs permet à un administrateur de voir les informations de chaque utilisateurs (avec filtre par identifiant possible), de le supprimer ou de lui donner ou retirer les privilèges administrateur.

Mes jeux favoris fonctionne de la même manière que la page de liste des jeux, sans le module de filtre et n'affiche que les jeux ayant appartenant à la liste des favoris de l'utilisateur.

Gestion des horaires permet à un administrateur de mettre à jour les horaires d'ouverture et de fermeture de la structure.

Ce paramètre influe sur les restrictions lors de la création d'une table.

Les jours de fermeture, les deux heures sont renseignée à 00 :00.



L'onglet Nouveau jeu donne accès au formulaire de création. Tous les champs sont requis.

Les champs numériques sont limités à la saisie de chiffres. De plus, une validation côté serveur garantit que l'image est bien fournie, les champs ont un format correct et le nom du jeu est bien unique.

Le champs de description utilise l'éditeur de texte riche Quill pour permettre à l'administrateur de mettre en forme le texte.

Une fois le formulaire soumis, les données sont envoyées au backend via une requête POST à l'API. En cas de succès, le jeu est ajouté à la base de données et l'administrateur est redirigé. En cas d'erreur, un message d'erreur s'affiche.

4. Optimisation pour les moteurs de recherche (SEO) et accessibilité

Bien que la priorité du projet TGMT ait été la mise en place des fonctionnalités de base, une attention particulière a été portée à l'optimisation pour les moteurs de recherche (SEO) et à l'accessibilité, afin de rendre l'application utilisable par le plus grand nombre et de favoriser sa visibilité.

a. Optimisation pour les moteurs de recherche (SEO) :

Plusieurs bonnes pratiques SEO ont été mises en œuvre :

- **Structure HTML sémantique :** L'application utilise des balises HTML sémantiques (<header>, <footer>, <nav>, <main>, <section>, <article>, <h1>, <h2>, etc.) pour structurer le contenu de manière logique et hiérarchique. Cette structuration sémantique facilite l'indexation du site par les moteurs de recherche, en leur permettant de mieux comprendre le contenu de chaque page.
- **Balises Meta :** Chaque page de l'application inclut une balise <title> descriptive et unique, essentielle pour le référencement. De plus, une balise <meta name="description"> pourrait être ajoutée pour fournir un résumé concis du contenu de chaque page (cette fonctionnalité n'est pas implémentée dans la version actuelle, mais est prévue dans les évolutions futures).
- **Attributs alt pour les images :** Des textes alternatifs descriptifs sont fournis pour toutes les images pertinentes de l'application via l'attribut alt. Ces textes alternatifs sont utilisés par les moteurs de recherche pour comprendre le contenu des images, et sont également utiles pour l'accessibilité (lecture d'écran).
- **URLs sémantiques :** Les URLs des différentes pages de l'application sont structurées de manière logique et significative (par exemple, /single-game/123 pour la page de détails d'un jeu, /tables pour la liste des tables). Cela améliore le référencement et la compréhension de la structure du site par les moteurs de recherche.
- **Sitemap :** Un plan du site (sitemap) a été conçu pour faciliter la navigation et l'indexation par les moteurs de recherche.
- **Balises de titre** La sémantique des balises de titre à été respectée pour une meilleur lecture du site par les robots.

b. Accessibilité :

L'accessibilité web a été une préoccupation importante lors du développement de l'application TGMT. Bien que toutes les recommandations WCAG (Web Content Accessibility Guidelines) n'aient pas pu être implémentées dans le cadre de ce projet, plusieurs mesures ont été prises pour améliorer l'accessibilité de l'application :

- **Responsive Design :** Comme mentionné précédemment, l'application est entièrement responsive, s'adaptant aux différentes tailles d'écran (ordinateur, tablette, smartphone). Cela garantit une expérience utilisateur correcte sur tous les appareils, y compris pour les utilisateurs de technologies d'assistance qui peuvent avoir des configurations d'écran spécifiques.
- **Contraste des couleurs :** Une attention particulière a été portée au contraste des couleurs, afin de garantir une lisibilité optimale pour tous les utilisateurs, y compris les personnes malvoyantes ou daltoniennes. Les couleurs utilisées respectent les ratios de contraste minimum recommandés par les WCAG (niveau AA).
- **Texte alternatif pour les images :** Comme mentionné plus haut, toutes les images pertinentes de l'application sont dotées d'un attribut alt descriptif, permettant aux utilisateurs de lecteurs d'écran de comprendre le contenu de l'image.
- **Placeholders pour les champs de formulaire :** Des placeholders ont été utilisés dans les champs de formulaire pour indiquer aux utilisateurs le type d'information attendu, améliorant ainsi l'ergonomie et la compréhension des formulaires.
- **Structure sémantique du HTML :** L'utilisation de balises HTML sémantiques (<header>, <footer>, <nav>, <main>, <article>, <h1>, <h2>, etc.) contribue à une meilleure structuration du contenu, facilitant la navigation pour les utilisateurs de technologies d'assistance.

Des pistes d'amélioration sont prévues pour renforcer l'accessibilité de l'application :

- **Attributs ARIA :** L'ajout d'attributs ARIA (Accessible Rich Internet Applications) permettra d'améliorer la sémantique des éléments interactifs et de fournir des informations supplémentaires aux lecteurs d'écran.
- **Thème "dys-friendly" :** La mise en place d'un thème spécifique pour les personnes dyslexiques est envisagée. L'utilisation de variables CSS pour la gestion des couleurs et des polices facilitera grandement l'implémentation de ce type de thème.
- **Navigation au clavier :** Une attention particulière sera portée à la navigation au clavier, pour s'assurer que tous les éléments interactifs sont accessibles sans l'utilisation d'une souris.

Réalisation back-end

Le backend de l'application TGMT est responsable de la gestion des données (jeux, tables, utilisateurs, horaires), de la logique métier (réservations, inscriptions, etc.), et de l'exposition d'une API RESTful pour permettre au frontend Angular d'interagir avec ces données. Il a été développé avec Node.js, en utilisant le framework Express, et la base de données MongoDB.

Le choix de Node.js s'est imposé pour plusieurs raisons. Tout d'abord, il permet d'utiliser JavaScript à la fois pour le frontend et le backend, ce qui simplifie le développement et la maintenance. De plus, Node.js est réputé pour sa rapidité et sa capacité à gérer un grand nombre de connexions simultanées, ce qui est important pour une application web. Express, un framework minimaliste pour Node.js, a été choisi pour sa simplicité et sa flexibilité, permettant de créer rapidement une API RESTful. Enfin, MongoDB, une base de données NoSQL orientée document, a été sélectionnée pour sa flexibilité et son adéquation avec le type de données manipulées par l'application (données non structurées, comme les informations sur les jeux).

Cette section présente en détail les réalisations backend de l'application. Nous commencerons par décrire la structure du code (contrôleurs, modèles, routes, middlewares) et l'organisation des fichiers. Ensuite, nous nous concentrerons sur les *points clés* de l'implémentation, en fournissant des extraits de code commentés pour illustrer :

- La définition des schémas Mongoose et l'intéraction avec la base.
- La logique de création, modification et suppression des tables de jeu, avec les *validations* associées (horaires, chevauchements, nombre de participants, etc.).
- La logique de création, modification et suppression des jeux.
- La logique de gestion des inscriptions et désinscriptions aux tables.
- La logique d'authentification (gestion des utilisateurs, hachage des mots de passe, génération et vérification des tokens JWT).
- La logique de gestion des utilisateurs, et des horaires d'ouverture.

Une attention particulière sera portée à la *sécurité* et à la *gestion des erreurs*, deux aspects essentiels du développement backend.

1. Structure du code

Le code source du backend de l'application TGMT est organisé selon une architecture MVC (Modèle-Vue-Contrôleur), un modèle de conception logicielle couramment utilisé pour le développement d'applications web. Cette architecture permet de séparer clairement les différentes responsabilités : la gestion des données (modèles), la logique de traitement (contrôleurs), et l'interaction avec le client (pas de vues, dans le cas d'une API RESTful, mais des *routes*).

Le code est structuré en plusieurs dossiers principaux :

- **controllers :** Ce dossier contient les *contrôleurs*. Chaque contrôleur est responsable d'une ressource spécifique (jeux, tables, utilisateurs, horaires). Les contrôleurs contiennent les méthodes qui gèrent les requêtes HTTP (GET, POST, PUT, DELETE) pour chaque route. Ils interagissent avec les modèles pour accéder aux données, et renvoient les réponses au client (au frontend).
- **models :** Ce dossier contient les modèles Mongoose. Chaque modèle définit la structure d'un document dans une collection MongoDB et fournit des méthodes pour interagir avec cette collection (créer, lire, mettre à jour, supprimer des documents).
- **routes :** Ce dossier contient les fichiers qui définissent les routes de l'API RESTful. Chaque fichier de route correspond à une ressource (jeux, tables, utilisateurs) et associe les différentes méthodes HTTP (GET, POST, PUT, DELETE) à des méthodes des contrôleurs.
- **middlewares :** Ce dossier contient les middlewares : les fonctions qui s'exécutent avant les contrôleurs, et qui peuvent effectuer des tâches telles que l'authentification, la validation des données, etc.
- **bin:** Ce dossier contient le script qui permet de lancer le serveur (www)

Le fichier principal de l'application backend est app.js. Ce fichier initialise l'application Express, configure les middlewares (CORS, body-parser, etc.), se connecte à la base de données MongoDB, importe les routes, et démarre le serveur.

2. Base de données

L'application TGMT utilise MongoDB, une base de données NoSQL orientée document, pour le stockage des données.

Ce choix a été motivé par plusieurs facteurs, qui répondent spécifiquement aux besoins du projet :

- **Flexibilité :** MongoDB offre une grande flexibilité en termes de structure des données. Contrairement aux bases de données relationnelles, il n'est pas nécessaire de définir un schéma rigide à l'avance. Cela facilite l'ajout de nouvelles propriétés aux documents (par exemple, l'ajout de nouvelles caractéristiques pour les jeux) sans avoir à modifier la structure de la base de données. Cette flexibilité était particulièrement intéressante pour le projet TGMT, car les jeux de société peuvent avoir des caractéristiques très variables (règles complexes, matériel spécifique, etc.).
- **Adapté aux données non structurées :** Les données manipulées par l'application (en particulier les informations sur les jeux) ne sont pas *strictement* structurées. Un jeu peut avoir des propriétés spécifiques (présence de figurines, type de dés, etc.) qui ne sont pas communes à tous les jeux. MongoDB, avec son modèle de données flexible basé sur des documents JSON, est bien adapté à ce type de données.
- **Facilité d'utilisation avec Mongoose :** La librairie Mongoose, un ODM (Object Document Mapper) pour MongoDB, facilite grandement l'interaction avec la base de données depuis Node.js. Elle permet de définir des schémas (pour avoir un minimum de structure), de valider les données, et d'effectuer des opérations CRUD (Create, Read, Update, Delete) de manière simple et intuitive.
- **Scalabilité :** MongoDB est conçu pour être scalable, c'est-à-dire qu'il peut gérer de grandes quantités de données et de trafic. Bien que ce ne soit pas un critère *essentiel* pour un projet de cette taille, c'est un avantage à long terme si l'application est amenée à évoluer.
- **Performance :** MongoDB est réputé pour ses performances, en particulier pour les opérations de lecture.

- **Popularité :** MongoDB est une base de données NoSQL très populaire, ce qui signifie qu'il existe une grande communauté d'utilisateurs et de nombreuses ressources disponibles en ligne.

Les données de l'application sont organisées en quatre collections principales :

- **users :** Contient les informations sur les utilisateurs (nom d'utilisateur, nom, prénom, email, mot de passe haché, rôle, etc.).
- **games :** Contient les informations sur les jeux (nom, description, type, catégorie, image, règles, etc.).
- **gametables :** Contient les informations sur les tables de jeu (jeu associé, créateur, date et heure de début, date et heure de fin, participants, etc.).
- **openinghours :** Contient les informations sur les horaires d'ouverture et de fermeture.

Les schémas Mongoose correspondants, qui définissent la structure de ces collections, sont présentés en annexe(6).

Les relations entre les différentes collections MongoDB sont illustrées dans le schéma de base de donnée et le diagramme de classes UML présenté en Annexe(7). On peut y distinguer les relations suivantes :

Création de tables : Un utilisateur (User) peut créer plusieurs tables de jeu (GameTable). Cette relation est représentée par le champ creator dans le modèle GameTable, qui contient l'ID de l'utilisateur créateur. Il s'agit d'une relation un-à-plusieurs (un utilisateur peut créer plusieurs tables, mais une table n'a qu'un seul créateur).

Participation aux tables : Plusieurs utilisateurs (User) peuvent participer à plusieurs tables de jeu (GameTable). Cette relation est modélisée par le champ participants dans le modèle GameTable, qui est un tableau contenant les ID des utilisateurs inscrits. Il s'agit d'une relation plusieurs-à-plusieurs.

Association jeu-table : Une table de jeu (GameTable) est associée à un et un seul jeu (Game). Cette relation est représentée par le champ game dans le modèle GameTable, qui contient l'ID du jeu associé. Il s'agit d'une relation plusieurs-à-un (plusieurs tables peuvent être associées à un même jeu, mais une table est toujours liée à un seul jeu).

Horaires : Il existe une seule entité OpeningHours.

3. API RESTFul

L'application TGMT expose une API RESTful pour permettre au frontend Angular d'interagir avec les données du backend (jeux, tables, utilisateurs, horaires). Cette API est construite en suivant les principes de l'architecture REST (Representational State Transfer), utilisant les verbes HTTP standard (GET, POST, PUT, DELETE) pour effectuer des opérations sur les ressources.

Une description complète et détaillée de toutes les routes de l'API, incluant les méthodes HTTP, les URLs, les descriptions, les paramètres (query et body), les formats des données, et les codes de retour (succès et erreurs), est fournie en Annexe (5).

L'API peut être regroupée en plusieurs catégories de routes :

- **Gestion des jeux :** Routes pour récupérer la liste des jeux (avec options de recherche et de filtrage), récupérer les détails d'un jeu spécifique, créer un nouveau jeu, mettre à jour un jeu existant, et supprimer un jeu.
- **Gestion des tables :** Routes pour récupérer la liste des tables (avec options de filtrage), récupérer les détails d'une table spécifique, créer une nouvelle table, mettre à jour une table existante, supprimer une table, s'inscrire à une table, et se désinscrire d'une table.
- **Gestion des utilisateurs :** Routes pour s'inscrire, se connecter, récupérer les informations de l'utilisateur connecté, mettre à jour les informations de l'utilisateur, supprimer un utilisateur, et gérer les rôles des utilisateurs (pour les administrateurs).
- **Gestion des favoris :** Routes pour ajouter un jeu aux favoris, retirer un jeu des favoris, récupérer la liste des favoris d'un utilisateur, et vérifier si un jeu est dans les favoris d'un utilisateur.
- **Gestion des horaires** Routes pour récupérer et modifier les horaires d'ouverture.

L'authentification est gérée via des tokens JWT (JSON Web Token), envoyés dans l'en-tête *Authorization* des requêtes. Les routes qui nécessitent une authentification sont protégées par un middleware *auth*. Certaines routes sont réservées aux administrateurs.

4. Composants métier :

Les contrôleurs constituent le cœur de la logique métier de l'application TGMT. Ils reçoivent les requêtes HTTP, interagissent avec les modèles Mongoose pour accéder aux données, effectuent les traitements nécessaires, et renvoient les réponses appropriées au frontend. Cette section présente les méthodes clés des contrôleurs, en mettant en évidence les aspects les plus significatifs de leur implémentation.

a. gameController.js :

Ce contrôleur gère les opérations liées aux jeux. Il expose des méthodes pour créer un nouveau jeu (createGame), mettre à jour un jeu existant (updateGame), supprimer un jeu (deleteGame), récupérer la liste de tous les jeux (listGames), rechercher des jeux avec des filtres (searchGames) et récupérer un jeu par son ID (getGameById). Nous nous concentrerons ici sur les méthodes createGame et updateGame, qui illustrent la gestion des données, la validation, et l'interaction avec le modèle Mongoose Game.

```
createGame: (req, res) => {
    if (!req.file) {
        return res.status(500).json({
            status: 500,
            message: 'Image is required',
        });
    }
    var game = req.body;
    delete game._id;

    var newGame = new Game({
        ...game,
        image: `${req.protocol}://${req.get('host')}/images/
        // ${req.file.filename}`
    });

    newGame.save()
        .then(savedGame => {
            res.status(201).json({
                status: 201,
                message: 'Game created successfully',
                result: savedGame,
            });
        })
        .catch(error => {
            res.status(500).json({
                status: 500,
                message: 'Error when creating game',
                error: error.message,
            });
        });
},
```

Cet extrait montre les étapes clés de la création d'un jeu. La présence d'un fichier image est vérifiée. Un nouvel objet *Game* est créé à partir des données de la requête *req.body* et du chemin relatif de l'image uploadée. La méthode *save()* de Mongoose est utilisée pour persister le nouveau jeu en base de données. Les erreurs sont gérées via les codes de retour HTTP appropriés (400 pour une requête incorrecte, 500 pour une erreur serveur).

```
updateGame: (req, res) => {
    const gameId = req.params.id;
    let gameData = req.body;
    if (req.file) {
        Game.findById(gameId)
            .then(foundGame => {
                if (!foundGame) {
                    return res.status(404).json({
                        status: 404,
                        message: 'Game not found',
                    });
                }
                if (foundGame.image) {
                    const oldImagePath = path.join(__dirname, '..', 'public',
                        'images', path.basename(foundGame.image));
                    if (fs.existsSync(oldImagePath)) {
                        fs.unlinkSync(oldImagePath);
                        console.log(`Ancienne image supprimée : ${oldImagePath}`);
                    }
                }
                gameData.image = `${req.protocol}://${req.get('host')}/images/${req.file.filename}`;
                return Game.findByIdAndUpdate(gameId, gameData, { new: true });
            })
            .then(updatedGame => {
                if (!updatedGame) {
                    return res.status(404).json({
                        status: 404,
                        message: 'Game not found',
                    });
                }
                res.status(200).json({
                    status: 200,
                    message: 'Game updated successfully',
                    result: updatedGame,
                });
            })
            .catch(error => {
                console.error('Erreur lors de la mise à jour du jeu :', error);
                res.status(500).json({
                    status: 500,
                    message: 'Error when updating game',
                    error: error.message,
                });
            });
    } else {
        Game.findByIdAndUpdate(gameId, gameData, { new: true })
            .then(updatedGame => {…
            })
            .catch(error => {…
            });
    }
},
```

Lors de la mise à jour d'un jeu, si une nouvelle image est fournie, l'ancienne image est supprimée du système de fichiers (si elle existe et est différente), et le chemin de la nouvelle image est enregistré. Si aucune nouvelle image n'est fournie, les autres données du jeu sont mises à jour.

La méthode *findByIdAndUpdate* de Mongoose est utilisée, avec l'option *{ new: true }* pour renvoyer le document mis à jour.

b. tableController.js :

Ce contrôleur gère les opérations liées aux tables de jeu, incluant la création, la modification, la suppression, la consultation, l'inscription et la désinscription. Nous allons nous concentrer ici sur la méthode joinTable, qui illustre la logique d'inscription d'un utilisateur à une table, avec les vérifications nécessaires.

```
const joinTable = async (req, res) => {
    const tableId = req.params.id;
    const userId = req.user.userId;
    try {
        const table = await GameTable.findById(tableId);
        if (!table) {
            return res.status(404).json({ error: 'Table de jeu non trouvée.' });
        }
        if (!table.open) {
            return res.status(400).json({ error: 'Les inscriptions à cette table sont fermées.' });
        }
        if (table.participants.includes(userId)) {
            return res.status(400).json({ error: 'Vous êtes déjà inscrit à cette table.' });
        }
        if (table.participants.length >= table.maxParticipants) {
            return res.status(400).json({ error: 'Le nombre maximum de participants est atteint.' });
        }
        table.participants.push(userId);
        await table.save();
        res.status(200).json({ message: 'Inscription réussie.' });
    } catch (err) {
        console.error("Erreur lors de l'inscription à la table de jeu :", err);
        res.status(500).json({ error: "Erreur lors de l'inscription à la table de jeu." });
    }
};
```

Cette méthode permet à un utilisateur connecté de s'inscrire à une table de jeu existante. Elle reçoit l'ID de la table en paramètre d'URL (req.params.id) et récupère l'ID de l'utilisateur connecté à partir du token JWT.

Nous illustrons ici plusieurs points clés du développement backend :

- **Récupération des données :** L'ID de la table est récupéré depuis les paramètres de l'URL, et l'ID de l'utilisateur depuis l'objet *req.user* (ajouté par le middleware d'authentification).
- **Validation des données :** La méthode effectue plusieurs vérifications *avant* d'autoriser l'inscription :
    - La table existe-t-elle ? (404 Not Found si non)
    - La table est-elle ouverte aux inscriptions ? (400 Bad Request si non)
    - L'utilisateur est-il déjà inscrit ? (400 Bad Request si oui)
    - La table est-elle complète ? (400 Bad Request si oui)

  Ces validations garantissent l'intégrité des données.

- **Interaction avec la base de données :** La méthode utilise *GameTable.findById* pour rechercher la table, puis *table.participants.push(userId)* et *table.save()* pour ajouter l'utilisateur à la liste des participants et enregistrer la modification en base de données. Cela illustre l'utilisation de Mongoose pour interagir avec MongoDB.
- **Gestion des erreurs :** Un bloc try...catch gère les erreurs potentielles. En cas d'erreur, un message d'erreur est consigné dans la console, et une réponse d'erreur 500 (Internal Server Error) est renvoyée au client.
- **Réponse au client :** En cas de succès, une réponse 200 OK est renvoyée, avec un message de confirmation.

c. userController.js :

Ce contrôleur gère les opérations liées aux utilisateurs, incluant l'inscription, la connexion, la mise à jour des informations, la suppression de compte, ainsi que la gestion des rôles pour les administrateurs. Nous nous intéresserons ici à la méthode *login*, qui illustre le processus d'authentification, incluant le hachage de mot de passe avec bcrypt et la génération de tokens JWT.

```
login: (req, res) => {
    User.findOne({ email: req.body.email })
        .then((user) => {
            if (!user) {
                return res.status(401).json({
                    status: 401,
                    message: 'Authentication failed',
                });
            }

            bcrypt.compare(req.body.password, user.password)
                .then((valid) => {
                    if (!valid) {
                        return res.status(401).json({
                            status: 401,
                            message: 'Wrong password',
                        });
                    }

                    const token = jwt.sign(
                        { userId: user._id },
                        process.env.SECRET,
                        { expiresIn: '24h' }
                    );

                    return res.status(200).json({
                        status: 200,
                        message: 'Authentication successful',
                        userId: user._id,
                        token: token,
                        role: user.role,
                    });
                })
                .catch((error) => {
                    return res.status(500).json({
                        status: 500,
                        message: 'Error when comparing passwords',
                        error: error.message,
                    });
                });
        })
        .catch((error) => {
            return res.status(404).json({
                status: 404,
                message: 'User not found',
                error: error.message,
            });
        });
},
```

La méthode reçoit l'email et le mot de passe dans le corps de la requête POST.

Recherche de l'utilisateur : *findOne* de Mongoose est utilisée pour rechercher un utilisateur correspondant à l'email.

Vérification de l'utilisateur : Si aucun utilisateur n'est trouvé avec cet email, une erreur 401 est renvoyée.

Comparaison des mots de passe : Si un utilisateur est trouvé, la méthode *bcrypt.compare* compare le mot de passe fourni avec le mot de passe haché stocké en base de données.

Vérification du mot de passe : Si les mots de passe ne correspondent pas, une erreur 401 est renvoyée.

Génération du token JWT : Si les mots de passe correspondent, un token JWT est généré avec *jwt.sign*. Ce token contient l'ID de l'utilisateur et son rôle, et il a une durée de validité de 24 heures. La clé secrète utilisée pour signer le token est stockée dans une variable d'environnement (*process.env.SECRET*).

Renvoie des informations utilisateur: le token, mais aussi le role, et le userId

Gestion des erreurs : Si une erreur se produit lors de la comparaison des mots de passe, une erreur 500 (Internal Server Error) est renvoyée. Si aucun utilisateur n'est trouvé, une erreur 404 est renvoyée.

d. openingHoursController.js :

Ce contrôleur gère les opérations liées aux horaires d'ouverture du bar à jeux. Il permet de récupérer les horaires (getOpeningHours) et de les mettre à jour (updateOpeningHours). La méthode updateOpeningHours est protégée et réservée aux administrateurs.

Eléments de sécurité

La sécurité de l'application TGMT a été une préoccupation centrale tout au long du processus de développement. Plusieurs mesures ont été mises en place, à la fois côté frontend (Angular) et côté backend (Node.js/Express, MongoDB), pour protéger les données des utilisateurs et prévenir les attaques courantes.

1. Authentification et autorisation :

- Authentification avec JWT : L'authentification des utilisateurs est gérée via des tokens JWT (JSON Web Token). Le processus d'authentification, ainsi que la génération et la vérification des tokens, ont été décrits avec un extrait de code de la méthode login du contrôleur userController.js.

- Hachage des mots de passe : Les mots de passe des utilisateurs sont hachés avec l'algorithme bcrypt avant d'être stockés en base de données, comme expliqué dans la section III.B.2. Cela garantit qu'en cas de compromission de la base de données, les mots de passe ne sont pas accessibles en clair.

- Protection des routes :

> **- Middleware auth (Backend) :** Un middleware d'authentification (auth.js) vérifie la présence et la validité du token JWT pour toutes les routes nécessitant une authentification.
>
> **- Guards Angular (AuthGuard et AdminGuard) :** Côté frontend, des guards Angular (AuthGuard et AdminGuard) sont utilisés pour protéger les routes et n'autoriser l'accès qu'aux utilisateurs connectés et, le cas échéant, disposant du rôle administrateur. Le fonctionnement de ces guards a été détaillé dans la section II.B.6 (Organisation du code - Guards).

2. Protection contre les failles XSS :

> **- Utilisation de DomSanitizer :** Pour prévenir les attaques par Cross-Site Scripting (XSS), l'application utilise le service DomSanitizer d'Angular pour "nettoyer" le contenu HTML potentiellement dangereux avant de l'afficher.

```
getSafeDescription(): SafeHtml | string {
  if (this.game && this.game.description) {
    return this.sanitizer.bypassSecurityTrustHtml(this.game.description);
  }
  return '';
}
```

3. Validation des données contre injection SQL (et NoSQL):

- Validation côté client (Angular) : Des validateurs Angular (required, min, max, pattern, etc.) sont utilisés dans les formulaires réactifs pour vérifier la validité des données saisies par l'utilisateur avant leur envoi au serveur.

- Validation côté serveur (Mongoose et contrôleurs) : Une validation complète des données est effectuée côté serveur, avant toute opération d'écriture en base de données. Cette validation inclut :

- La vérification des types de données, à l'aide des schemas mongoose.

- La vérification de la présence des champs obligatoires.

- La vérification des contraintes de taille (par exemple, longueur minimale/maximale des chaînes de caractères).

- La vérification des contraintes de format (par exemple, format d'email valide).

- La vérification des règles métier spécifiques (par exemple, le nombre maximum de participants à une table ne doit pas dépasser la capacité maximale du jeu).

- La vérification des horaires

4. Stockage sécurisé des informations sensibles :

- Variables d'environnement : Les informations sensibles, telles que la chaîne de connexion à la base de données et la clé secrète utilisée pour signer les tokens JWT, sont stockées dans des variables d'environnement, et ne sont jamais incluses directement dans le code source.

- Hachage des mots de passe : Les mots de passe ne sont jamais stocké en clair.

Jeux d’essai

Afin de garantir le bon fonctionnement de l'application TGMT, et de valider la conformité de son comportement par rapport aux spécifications fonctionnelles, une série de tests manuels a été effectuée. Les jeux d'essai les plus représentatifs, avec pour chacun le scénario de test, le résultat attendu, le résultat obtenu, et l'analyse des éventuels écarts sont disponibles en annexe (8).

Respect Du RGPD

Le Règlement Général sur la Protection des Données (RGPD) est une réglementation européenne qui encadre le traitement des données personnelles. L'application TGMT, dans sa version actuelle, respecte les principes fondamentaux du RGPD, et des mesures ont été prises pour minimiser les risques et garantir la protection des données des utilisateurs.

- **Minimisation des données :** L'application ne collecte que les données *strictement nécessaires* à son fonctionnement. Lors de l'inscription, les informations demandées sont limitées à :
    - Nom d'utilisateur (pseudonyme)
    - Prénom
    - Nom
    - Adresse e-mail (utilisée pour la connexion et la récupération de mot de passe)
    - Numéro de téléphone (utilisé pour la communication liée aux reservations, *pas* à des fins de marketing)
    - Mot de passe (haché, non stocké en clair)

Aucune donnée sensible (informations de paiement, etc.) n'est collectée.

- **Information des utilisateurs :** Les utilisateurs sont informés de l'utilisation de leurs données via la page "Politique de confidentialité" [Mettre un lien hypertexte vers la page en question, si ton document Word le permet]. Cette page est en cours d'élaboration, et une version complète, rédigée par un professionnel du droit, sera mise en place après la soutenance.
- **Droits des utilisateurs :**
    - **Droit d'accès et de rectification :** Les utilisateurs peuvent consulter et modifier leurs informations personnelles via la page "Mes informations".
    - **Droit à l'effacement :** Les utilisateurs peuvent supprimer leur compte, ce qui entraîne la suppression de leurs données personnelles et de leurs réservations (comme démontré dans la section "Tests").
    - **Droit d'opposition, de limitation, de portabilité :** Ces droits ne sont pas *explicitement* implémentés dans la version actuelle de l'application, mais ils sont *pris en compte* dans la conception. Par exemple, aucune donnée n'est utilisée à des fins de profilage ou de marketing.
- **Sécurité des données :** (Voir la section "Sécurité" pour les détails : hachage des mots de passe, JWT, etc.).
- **Cookies :** L'application n'utilise *que* des cookies strictement nécessaires à son fonctionnement (cookies de session, liés à l'authentification). Aucun cookie de suivi ou de publicité n'est utilisé. La mise en place d'un mécanisme de consentement explicite aux cookies (non essentiels) est prévue dans les évolutions futures.

Déploiement :

L'application TGMT est déployée sur la plateforme **Render**, qui offre des services d'hébergement pour les applications web (frontend et backend) et les bases de données. Render a été choisi pour sa *simplicité d'utilisation*, son *plan gratuit* adapté à un projet de démonstration, et sa *bonne intégration avec les technologies utilisées* (Node.js, Angular, MongoDB).

Le déploiement est structuré en deux services distincts :

- **Un service "Web Service" pour le backend (Node.js/Express) :** Ce service exécute le code du serveur, gère l'API RESTful, et interagit avec la base de données MongoDB Atlas.
- **Un "Static Site" pour le frontend (Angular) :** Ce service sert les fichiers statiques de l'application Angular (HTML, CSS, JavaScript, images).

Étapes du déploiement :

- Création d'un compte Render
- Création d'un Web Service pour le backend
- Création d'un Static Site pour le frontend
- Mise à jour de l'URL du backend dans le frontend :
    - L'application Angular a ensuite été reconstruite (ng build --configuration production) pour prendre en compte cette nouvelle URL, et les modifications ont été poussées sur le dépôt Git.
- Déploiement continu :
    - Render est configuré pour déployer automatiquement les nouvelles versions de l'application à chaque push sur la branche main du dépôt Git.

URL de l'application :

https://tgmt.onrender.com/

Veille durant le projet

Une veille technologique régulière a été effectuée afin de se tenir informé des dernières tendances, des bonnes pratiques, et des éventuelles mises à jour des technologies utilisées. Cette veille a porté sur les points suivants :

- **Angular :** Suivi des nouvelles versions d'Angular, des correctifs de sécurité, et des bonnes pratiques de développement. *Sources :* Documentation officielle d'Angular, blog officiel Angular, blog de Netapsys.
- **Node.js/Express :** Suivi des nouvelles versions de Node.js et d'Express, des correctifs de sécurité, et des bonnes pratiques via la documentation officielle de Node.js et d'Express.
- **MongoDB/Mongoose :** Suivi des nouvelles versions de MongoDB et de Mongoose, des correctifs de sécurité, et des bonnes pratiques. Connexion hebdomadaire au officielle de MongoDB.
- **JWT :** Suivi des bonnes pratiques en matière de sécurité des tokens JWT (durée de vie, algorithmes de signature, etc.). Sources : Documentation de la librairie jsonwebtoken et recommandations de l'OWASP (Open Web Application Security Project)

Conclusion

La réalisation de ce projet a été l'occasion de mettre en pratique l'ensemble des compétences acquises durant la formation "Les fondamentaux du développement" chez Holberton School. J'ai pu mettre en œuvre une architecture logicielle moderne, séparant distinctement le frontend (Angular) et le backend (Node.js/Express), et utiliser une base de données NoSQL (MongoDB) adaptée aux besoins du projet. La conception et le développement de l'application ont nécessité une approche méthodique, de la définition des besoins utilisateurs à la mise en place d'une API RESTful, en passant par la modélisation des données, la création d'interfaces utilisateur responsives et intuitives, et la sécurisation de l'application.

Au-delà des aspects purement techniques, ce projet m'a permis de développer des compétences transversales essentielles, telles que la gestion de projet, l'autonomie, la résolution de problèmes complexes, et la veille technologique. La nécessité de respecter des délais serrés et de produire un livrable de qualité professionnelle a été une expérience formatrice.

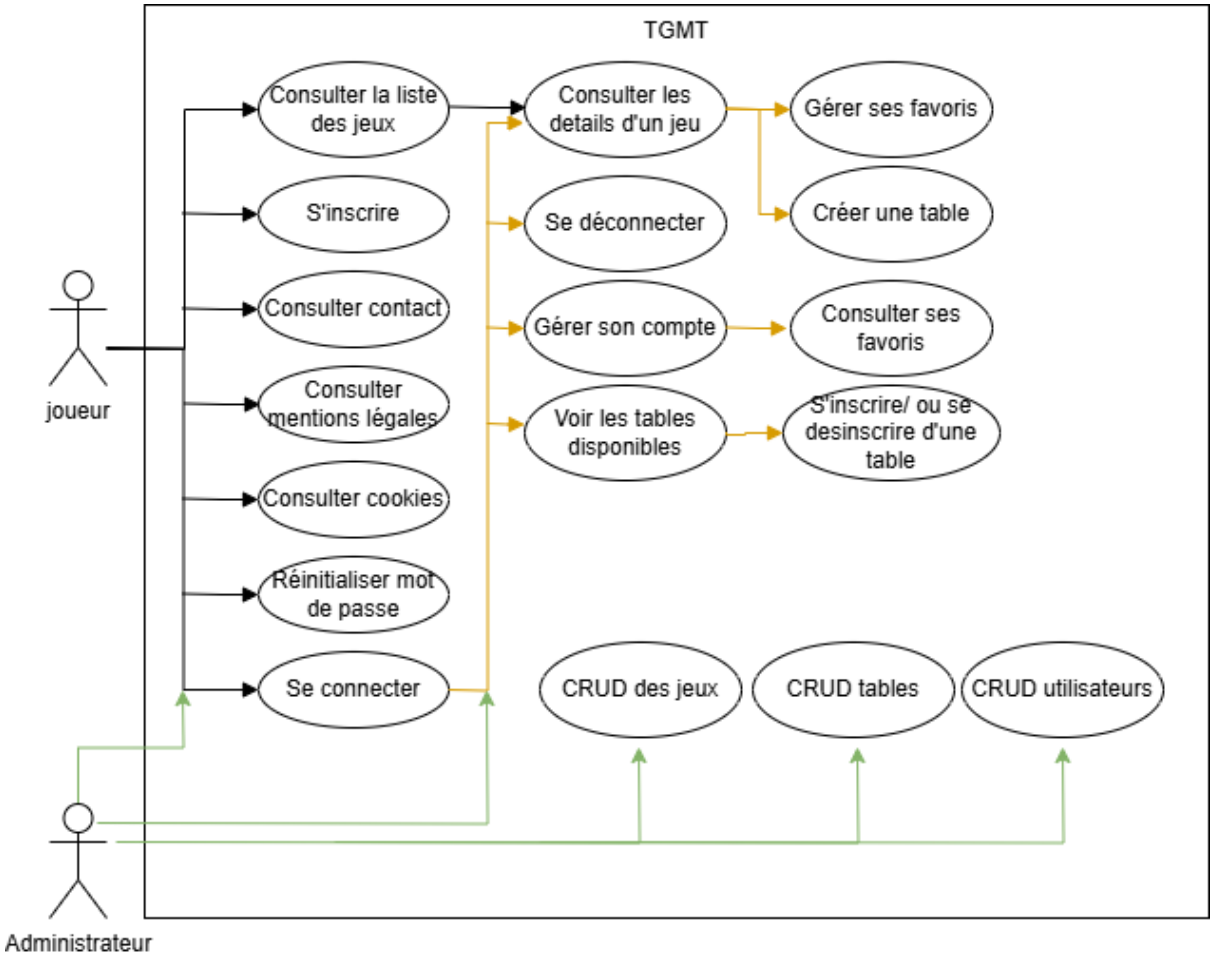
Bien que l'application TGMT soit pleinement fonctionnelle, de nombreuses évolutions sont envisageables pour enrichir l'expérience utilisateur et étendre les fonctionnalités. Parmi celles-ci, on peut citer :

- **L'implémentation d'un système de notifications** (email ou push) pour informer les utilisateurs des modifications concernant leurs réservations (confirmation, rappel, annulation, etc.).
- La mise en place d'un **système de file d'attente**
- **L'ajout d'un système de chat** intégré à chaque table de jeu, pour faciliter la communication entre les participants.
- **L'intégration d'un calendrier** pour visualiser les réservations de manière plus intuitive.

La structure modulaire de l'application, tant côté frontend (avec les composants Angular) que côté backend (avec l'architecture MVC et l'API RESTful), facilite grandement l'ajout de ces nouvelles fonctionnalités. Le projet TGMT constitue donc une base solide pour de futures évolutions, et démontre ma capacité à concevoir et développer des applications web complètes et évolutives.

Annexes

Annexe 1 : Diagramme des cas d'utilisation



Annexe 2 : Liste des librairies

Librairie Front-end	Version	Description
@angular/animations	^18.2.2	Module Angular pour la gestion des animations.
@angular/common	^18.2.2	Contient des directives, des pipes et des services communs, dont HttpClient pour les requêtes HTTP et CommonModule qui permet d'utiliser les directives *ngIf et *ngFor.
@angular/compiler	^18.2.2	Le compilateur Angular.
@angular/core	^18.2.2	Contient les fonctionnalités essentielles d'Angular (composants, services, injection de dépendances, etc.).
@angular/forms	^18.2.2	Module pour la gestion des formulaires (formulaires réactifs et template-driven).
@angular/platform-browser	^18.2.2	Module pour le rendu de l'application dans un navigateur.
@angular/platform-browser-dynamic	^18.2.2	Module pour la compilation et le rendu dynamique de l'application dans le navigateur.
@angular/router	^18.2.2	Module pour la gestion du routing (navigation entre les pages).
@ckeditor/ckeditor5-angular	^8.0.0	Composant Angular pour l'éditeur de texte riche CKEditor 5.
ngx-quill	^26.0.5	Wrapper Angular pour l'éditeur de texte riche Quill.
quill	^2.0.2	Éditeur de texte riche (utilisé par ngx-quill et/ou @ckeditor/ckeditor5-angular, à toi de choisir).
rxjs	~7.8.0	Librairie pour la programmation réactive (gestion des observables).
tslib	^2.3.0	Librairie d'aide pour TypeScript, contenant des fonctions utilitaires.
zone.js	~0.14.10	Librairie requise par Angular pour la détection des changements.

Librairie back-end	Version	Description
bcrypt	^5.1.1	Librairie pour le hachage des mots de passe (algorithme bcrypt).
connect-flash	^0.1.1	Permet l'affichage de message flash.
cookie-parser	~1.4.4	Middleware Express pour l'analyse des cookies HTTP.
cors	^2.8.5	Middleware Express pour la gestion des requêtes CORS (Cross-Origin Resource Sharing).
debug	~2.6.9	Outil de débogage pour Node.js.
dotenv	^16.4.5	Librairie pour charger les variables d'environnement depuis un fichier .env.
express	^4.19.2	Framework web minimaliste pour Node.js.
express-session	^1.18.0	Permet de gérer les sessions utilisateurs.
http-errors	~1.6.3	Librairie pour la création d'erreurs HTTP.
jsonwebtoken	^9.0.2	Librairie pour la création et la vérification de tokens JWT (JSON Web Tokens).
mongoose	^8.6.0	Librairie ODM (Object Document Mapper) pour MongoDB.
mongoose-unique-validator	^5.0.1	Plugin Mongoose pour la validation de l'unicité des champs.
morgan	~1.9.1	Middleware Express pour la journalisation (logging) des requêtes HTTP.
multer	^1.4.5-lts.1	Middleware Express pour la gestion des uploads de fichiers (multipart/form-data), utilisé pour les images des jeux.
nodemailer	^6.9.14	Librairie pour l'envoi d'e-mails depuis Node.js.

Annexe 3 : Liste des composants Angular

- **app.component.ts :** Le composant racine de l'application. Il contient généralement le squelette HTML de base (avec <router-outlet>) et la logique minimale.

- **header.component.ts :** Gère l'affichage de l'en-tête de l'application, y compris le logo, le menu de navigation (avec la logique pour le menu hamburger), et les liens de connexion/déconnexion/compte (en fonction de l'état de l'utilisateur).

- **footer.component.ts :** Gère l'affichage du pied de page de l'application (liens vers les mentions légales, la politique de cookies, etc.). Contenu principalement statique.

- **home.component.ts :** La page d'accueil de l'application. Affiche un message de bienvenue, une description de l'application, et des liens vers les fonctionnalités principales.

- **gamecard.component.ts :** Affiche la liste de *tous* les jeux, avec une barre de recherche et des options de filtrage. Utilise le composant game-card pour afficher chaque jeu.

- **single-game.component.ts :** Affiche les *détails* d'un jeu spécifique (description complète, règles, etc.), ainsi que la liste des *tables de jeu* disponibles pour ce jeu. Gère l'inscription/désinscription aux tables, et l'ajout/retrait des favoris.

- **create-table.component.ts :** Affiche le formulaire de *création* d'une table de jeu (accessible depuis la page de détails d'un jeu).

- **table-list.component.ts :** Affiche la liste de *toutes* les tables de jeu, avec des options de filtrage (par jeu, date/heure, statut).

- **signup.component.ts :** Gère le formulaire d'*inscription* d'un nouvel utilisateur.

- **signin.component.ts :** Gère le formulaire de *connexion* d'un utilisateur existant.

- **forgot-password.component.ts :** Gère le formulaire de demande de réinitialisation de mot de passe.

- **reset-password.component.ts :** Gère le formulaire de définition d'un nouveau mot de passe (après une demande de réinitialisation).

- **user-info.component.ts :** Permet à un utilisateur connecté de consulter et de modifier ses informations personnelles.

- **favorites.component.ts:** Affiche la liste des jeux favoris d'un utilisateur.

- **dashboard.component.ts :** Le tableau de bord.Contient des liens vers "Mes informations" et "Mes favoris". Pour les *administrateurs*, contient aussi des liens vers "Créer un jeu", "Gestion des utilisateurs", "Gestion des horaires".

- **create-game.component.ts :** (Dans un dossier admin) Gère le formulaire de *création* d'un nouveau jeu (réservé aux administrateurs).

- **update-game.component.ts:** Permet de modifier un jeu.

- **user-management.component.ts :** (Dans un dossier admin) Permet aux administrateurs de gérer les utilisateurs (liste, modification de rôle, suppression).

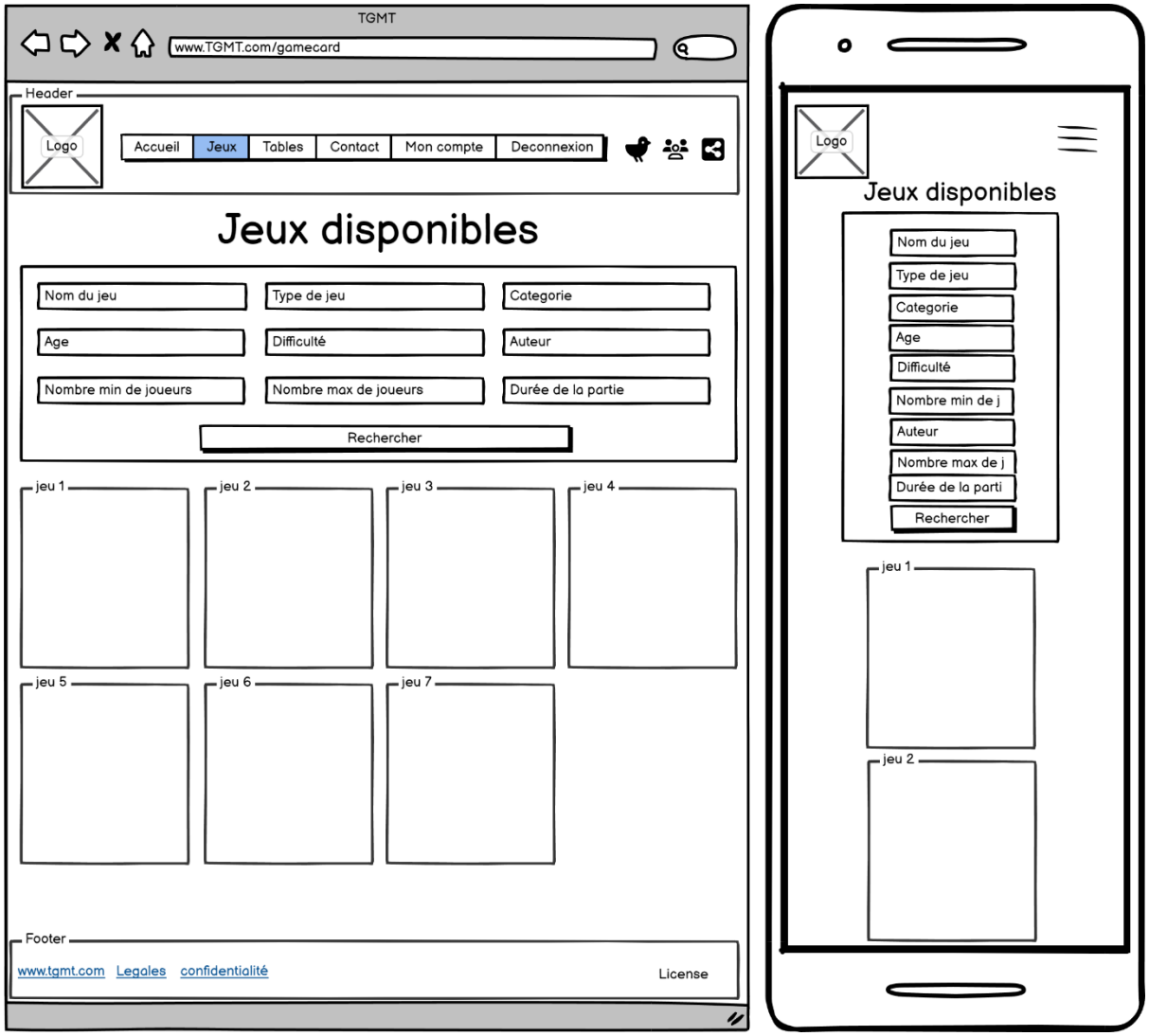
- **opening-hours.component.ts :** Permet à un administrateur de gérer les horaires d'ouverture du bar.

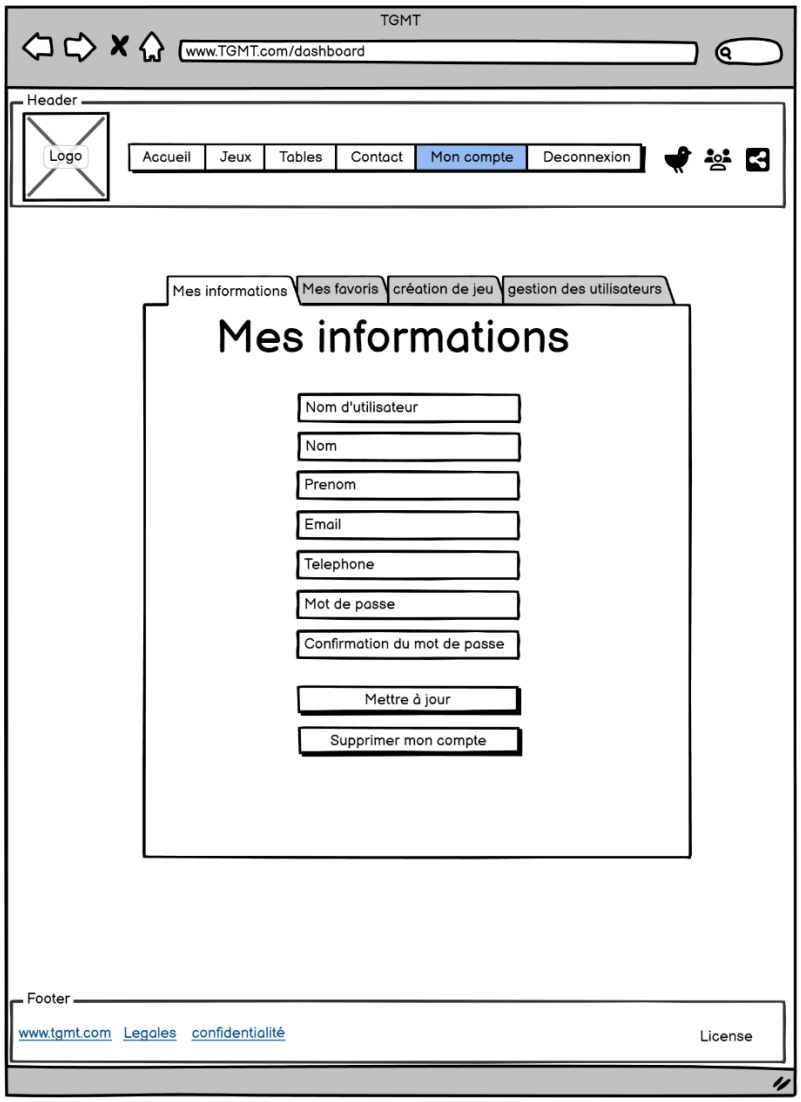
- **contact.component.ts :** Affiche la page de contact (principalement statique).

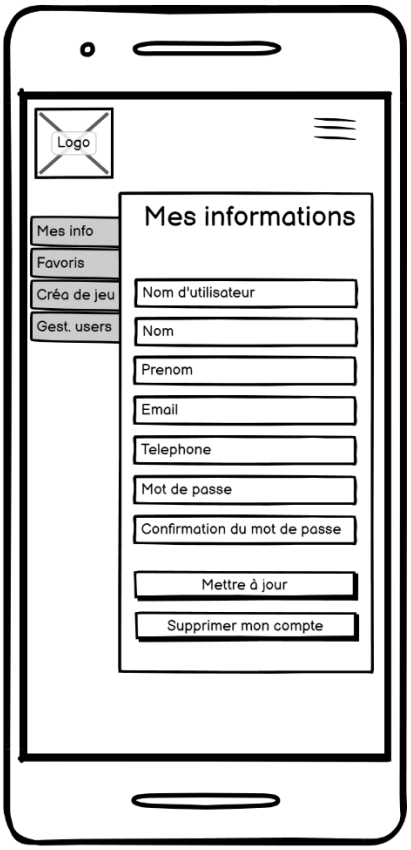
- **legal-notice.component.ts:** Affiche la page des mentions légales.

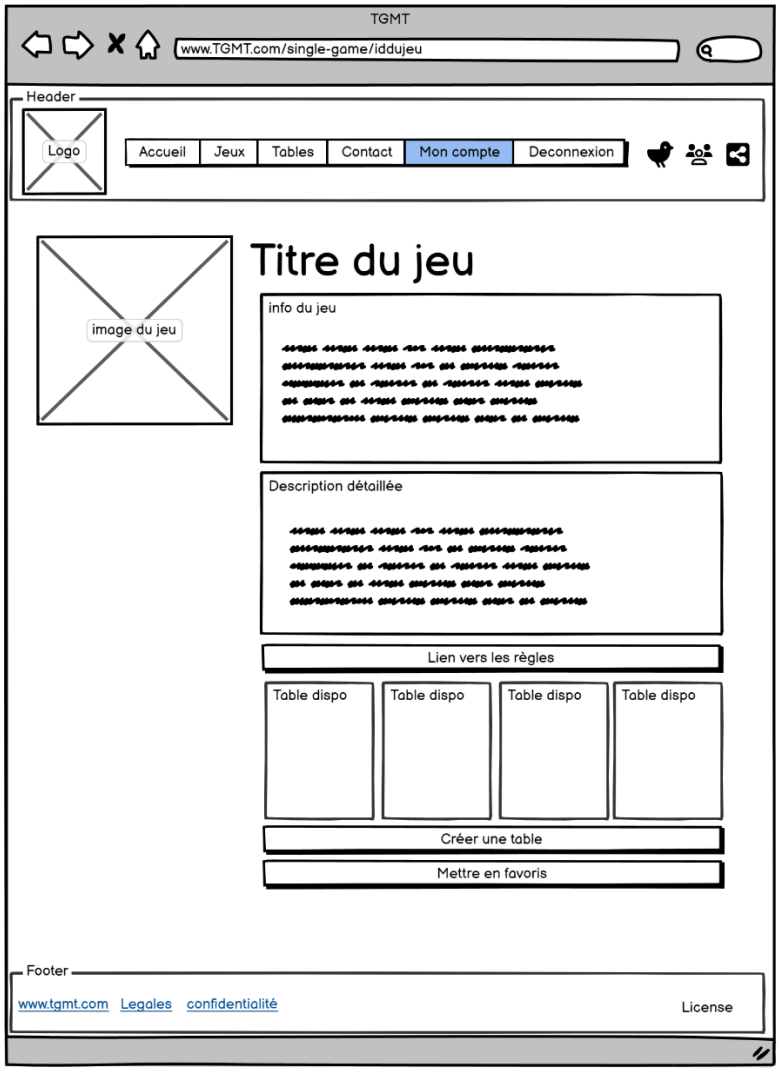
- **cookie-policy.component.ts:** Affihce la page des cookies.

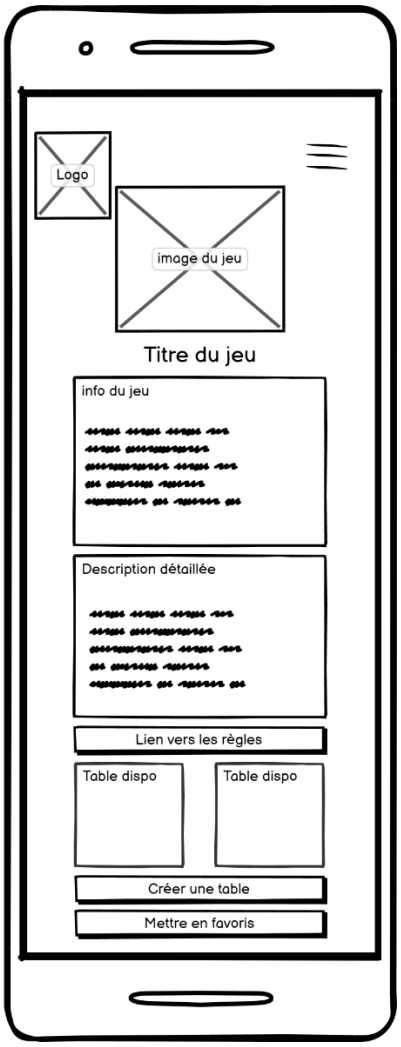
Annexe 4 : Maquettes




TGMT
www.TGMT.com/dashboard
Header
Logo
Accueil
Jeux
Tables
Contact
Mon compte
Deconnexion
Mes informations
Mes favoris
création de jeu
gestion des utilisateurs
Mes informations
Nom d'utilisateur
Nom
Prenom
Email
Telephone
Mot de passe
Confirmation du mot de passe
Mettre à jour
Supprimer mon compte
Footer
www.tgmt.com
Legales
confidentialité
License


Logo
Mes info
Favoris
Créa de jeu
Gest. users
Mes informations
Nom d'utilisateur
Nom
Prenom
Email
Telephone
Mot de passe
Confirmation du mot de passe
Mettre à jour
Supprimer mon compte


TGMT
www.TGMT.com/single-game/iddujeu
Header
Logo
Accueil
Jeux
Tables
Contact
Mon compte
Deconnexion
image du jeu
Titre du jeu
info du jeu
Description détaillée
Lien vers les règles
Table dispo
Table dispo
Table dispo
Table dispo
Créer une table
Mettre en favoris
Footer
www.tgmt.com
Legales
confidentialité
License


Logo
image du jeu
Titre du jeu
info du jeu
Description détaillée
Lien vers les règles
Table dispo
Table dispo
Créer une table
Mettre en favoris

Annexe 5 : tableau de l'API REST

Méthode	URL	Description	Paramètres (Query)	Paramètres (Route)	Corps de la requête (Body)	Réponse (Succès)	Réponse (Erreur)
GET	/api/games	Récupère la liste de tous les jeux.				200 OK [{ _id: string, name: string, type: string, category: string, ... , image: string }, ...]	500 Internal Server Error (erreur serveur)
GET	/api/games/search	Recherche des jeux en fonction de filtres.	name (string, optionnel), type (string, optionnel), category (string, optionnel), etc.			200 OK [{ game1 }, { game2 }, ...] (tableau d'objets jeux, potentiellement vide)	500 Internal Server Error
GET	/api/games/:id	Récupère les détails d'un jeu spécifique.		id (string)		200 OK { _id: string, name: string, ... }	404 Not Found (jeu non trouvé), 500 Internal Server Error
POST	/api/games	Crée un nouveau jeu (réservé aux administrateurs).			{ name: string, type: string, category: string, ageMin: number, difficulty: string, author: string, playerMin: number, playerMax: number, description: string, image: string, partytime: number, rules: string, resourcesLink: string[] }	201 Created { game: { ... } } (l'objet jeu créé, avec son _id)	400 Bad Request (données manquantes ou invalides), 401 Unauthorized (si pas admin), 500 Internal Server Error
PUT	/api/games/:id	Met à jour un jeu existant (réservé aux administrateurs).		id (string)	{ name?: string, type?: string, category?: string, ... , image?: string } (champs optionnels)	200 OK { game: { ... } } (l'objet jeu mis à jour)	400 Bad Request, 401 Unauthorized, 403 Forbidden (si pas admin), 404 Not Found (jeu non trouvé), 500 Internal Server Error
DELETE	/api/games/:id	Supprime un jeu (réservé aux administrateurs).		id (string)		200 OK { message: "Game deleted successfully" }	401 Unauthorized, 403 Forbidden, 404 Not Found, 500 Internal Server Error
GET	/api/tables	Récupère la liste de toutes les tables de jeu, avec des options de filtrage.	gameId (string, optionnel), startTime (string, optionnel), endTime (string, optionnel), available (string, optionnel)			200 OK [{ table1 }, { table2 }, ...]	500 Internal Server Error
POST	/api/tables	Crée une nouvelle table de jeu.			{ creator: string, startTime: string, game: string, endTime: string, duration: number, maxParticipants: number }	201 Created { gameTable: { ... } }	400 Bad Request (données invalides, chevauchement, etc.), 401 Unauthorized, 500 Internal Server Error
GET	/api/tables/:id	Récupère les détails d'une table de jeu spécifique.		id (string)		200 OK { gameTable: { ... } }	404 Not Found, 500 Internal Server Error
PUT	/api/tables/:id	Met à jour une table de jeu existante.		id (string)	{ startTime?: string, endTime?: string, duration?: number, maxParticipants?: number, open?: boolean, participants?: string[] } (champs optionnels)	200 OK { updatedTable: { ... } }	400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found, 500 Internal Server Error
DELETE	/api/tables/:id	Supprime une table de jeu.		id (string)		200 OK { message: "Table de jeu supprimée avec succès" }	401 Unauthorized, 403 Forbidden, 404 Not Found, 500 Internal Server Error
POST	/api/tables/:id/join	Inscrit l'utilisateur courant à une table de jeu.		id (string)	{} (corps vide)	200 OK { message: "Inscription réussie." }	400 Bad Request (table pleine, déjà inscrit, etc.), 401 Unauthorized, 404 Not Found, 500 Internal Server Error
POST	/api/tables/:id/leave	Désinscrit l'utilisateur courant d'une table de jeu.		id (string)	{} (corps vide)	200 OK { message: "Désinscription réussie." }	400 Bad Request (non inscrit), 401 Unauthorized, 404 Not Found, 500 Internal Server Error

Méthode	URL	Description	Paramètres (Query)	Paramètres (Route)	Corps de la requête (Body)	Réponse (Succès)	Réponse (Erreur)
POST	/users/login	Connecte un utilisateur.			{ email: string, password: string }	200 OK { userId: string, token: string, role: string }	401 Unauthorized (identifiants incorrects), 404 Not Found (utilisateur non trouvé), 500 Internal Server Error
POST	/users/signup	Inscrit un nouvel utilisateur.			{ username: string, firstname: string, lastname: string, email: string, password: string, telephone: string }	201 Created (pas de corps de réponse, ou un message de succès)	400 Bad Request (données invalides, utilisateur existe déjà), 500 Internal Server Error
GET	/users/current	Retourne les informations de l'utilisateur courant.			200 OK	401 Unauthorized, 404 Not Found, 500 Internal Server Error	
PUT	/users/current	Met à jour les informations de l'utilisateur connecté			{ username?: string, firstname?: string, lastname?: string, email?: string, telephone?: string, password?: string } (champs optionnels)	200 OK	400 Bad Request, 401 Unauthorized, 404 Not Found, 500 Internal Server Error
DELETE	/users/delete/:userId?	Supprime un utilisateur. L'ID est optionnel (si absent, supprime l'utilisateur courant).		userId (string, optionnel)		200 OK { message: "Compte utilisateur et données associées supprimés avec succès" }	401 Unauthorized, 403 Forbidden, 404 Not Found, 500 Internal Server Error
PUT	/users/:userId/update-role	Met à jour le rôle d'un utilisateur (admin seulement).		userId (string)	{ role: string }	200 OK	400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found, 500 Internal Server Error
GET	/users	Récupère la liste de tous les utilisateurs (admin seulement).				200 OK [{ user1 }, { user2 }, ...]	401 Unauthorized, 403 Forbidden, 500 Internal Server Error
GET	/users/search	Recherche un utilisateur	term (string)			200 OK	401 Unauthorized, 403 Forbidden, 500 Internal Server Error
POST	/users/forgot-password	Demande une réinitialisation de mot de passe.			{ email: string }	200 OK (pas de corps de réponse, ou un message de succès)	400 Bad Request (email manquant), 404 Not Found (utilisateur non trouvé), 500 Internal Server Error
POST	/users/reset-password/:token	Réinitialise le mot de passe.		token (string)	{ newPassword: string }	200 OK (pas de corps de réponse, ou un message de succès)	400 Bad Request (token invalide/expiré, mot de passe invalide), 500 Internal Server Error
GET	/api/opening-hours	Récupère les horaires d'ouverture.				200 OK (Objet contenant les horaires)	500 Internal Server Error
PUT	/api/opening-hours	Met à jour les horaires d'ouverture (admin seulement).			{ monday: { startTime: string, endTime: string }, ... } (objet avec les horaires pour chaque jour)	200 OK (Objet contenant les horaires mis à jour)	400 Bad Request, 401 Unauthorized, 403 Forbidden, 500 Internal Server Error
POST	/api/favorites/add-to-favorites	Ajoute un jeu aux favoris de l'utilisateur.			{ userId: string, gameId: string }	200 OK (pas de corps de réponse, ou un message de succès)	400 Bad Request, 401 Unauthorized, 500 Internal Server Error
POST	/api/favorites/remove-from-favorites	Retire un jeu des favoris de l'utilisateur.			{ userId: string, gameId: string }	200 OK	400 Bad Request, 401 Unauthorized, 500 Internal Server Error
GET	/api/favorites/:userId	Récupère la liste des jeux favoris d'un utilisateur.		userId (string)	200 OK	401 Unauthorized, 404 Not Found, 500 Internal Server Error	
POST	/api/favorites/is-favorite	Vérifie si un jeu est dans les favoris de l'utilisateur			{ userId: string, gameId: string }	200 OK { isFavorite: boolean }	400 Bad Request, 401 Unauthorized, 500 Internal Server Error

Annexe 6 : Modèles :

UserModele.js

```
const mongoose = require('mongoose');
const uniqueValidator = require('mongoose-unique-validator');

const Schema = mongoose.Schema;

var userSchema = new Schema({
    username: { type: String, required: false },
    firstname: { type: String, required: false },
    lastname: { type: String, required: false },
    email: { type: String, required: true, unique: true },
    password: { type: String, required: true },
    telephone: { type: String, required: false },
    role: { type: String, enum: ['user', 'admin'], default: 'user' },
    createdAt: {
        type: Date,
        default: Date.now
    },
    gameTables: [{ type: Schema.Types.ObjectId, ref: 'GameTable' }]
});

userSchema.plugin(uniqueValidator);

module.exports = mongoose.model('User', userSchema);
```

gameTableModel.js

```
const mongoose = require('mongoose');
const Schema = mongoose.Schema;

const gameTableSchema = new Schema({
    creator: { type: Schema.Types.ObjectId, ref: 'User', required: true },
    game: { type: Schema.Types.ObjectId, ref: 'Game', required: true },
    startTime: { type: Date, required: true, default: Date.now },
    endTime: { type: Date, required: true },
    duration: {type: Number, required: true},
    open: { type: Boolean, default: true },
    participants: [{ type: Schema.Types.ObjectId, ref: 'User' }],
    maxParticipants: { type: Number, required: true },
});


gameTableSchema.path('endTime').validate(function(value) {
    return this.startTime <= value;
}, 'La date et l\'heure de fin doivent être postérieures à la date et l\'heure de début.');

module.exports = mongoose.model('GameTable', gameTableSchema);
```

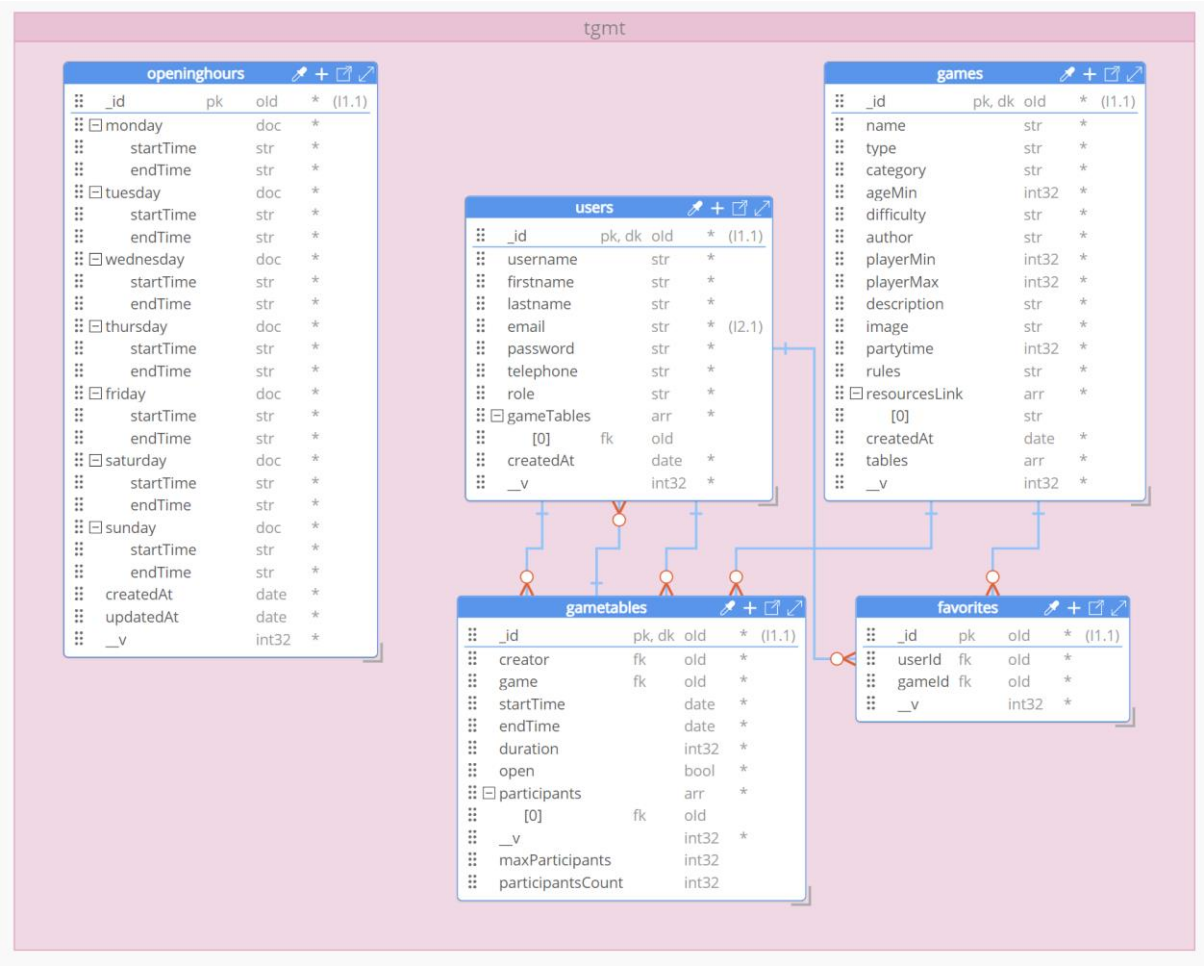
gameModel.js

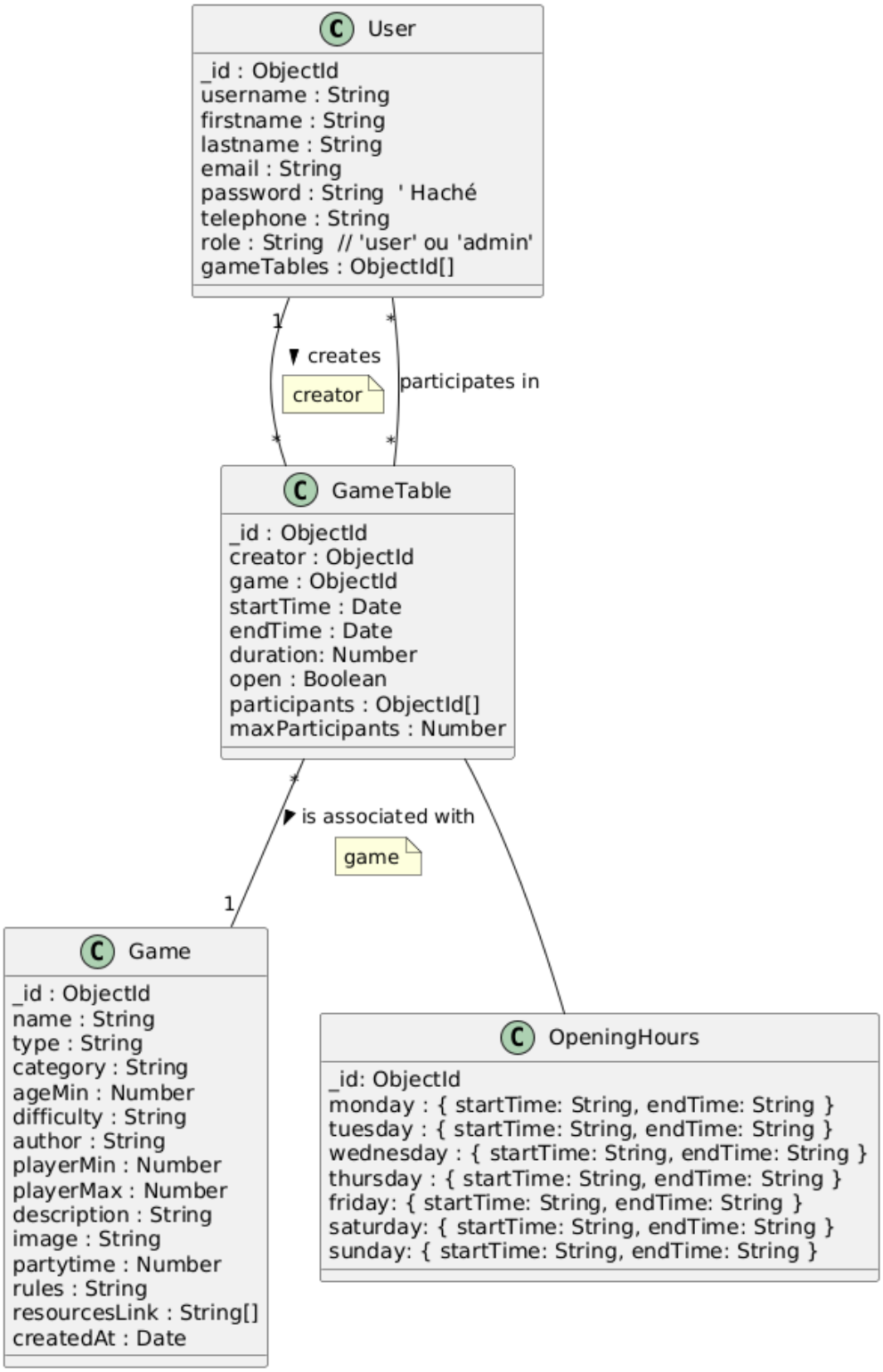
```
const { default: mongoose } = require("mongoose")
const schema = mongoose.Schema

var gameSchema = new schema({
    name: {type: String, required: true}, // Nom du jeu
    type: {type: String, required: true}, // Type de jeu (plateau, cartes, etc.)
    category: {type: String, required: true}, // Catégorie du jeu (stratégie, chance, etc.)
    ageMin: {type: Number, required: true}, // Âge minimum pour jouer
    difficulty: {type: String, required: true}, // Difficulté du jeu
    author: {type: String, required: true}, // Auteur du jeu
    playerMin: {type: Number, required: true}, // Nombre minimum de joueurs
    playerMax: {type: Number, required: true}, // Nombre maximum de joueurs
    description: {type: String, required: true}, // Description du jeu
    image: {type: String, required: true}, // URL de l'image du jeu
    partytime: {type: Number, required: true}, // Durée d'une partie en minutes
    rules: {type: String, required: true}, // Lien vers Règles du jeu
    resourcesLink: {type: Array},  // Liens vers des ressources externes (autres version des règles, vidéos, etc.)
    createdAt: { // Date de création de la fiche du jeu
        type: Date,
        default: Date.now
    },
    tables: [{ // Liste des tables de jeu créées pour ce jeu
        creator: { type: mongoose.Schema.Types.ObjectId, ref: 'User' }, // Référence à l'utilisateur qui a créé la table
        startTime: { type: Date, required: true }, // Date et heure de début de la table
        open: { type: Boolean, default: true }, // Indique si la table est ouverte aux inscriptions
        participants: [{ type: mongoose.Schema.Types.ObjectId, ref: 'User' }] // Liste des utilisateurs inscrits à la table
    }]

})

module.exports = mongoose.model('Game', gameSchema)
```

Favorite.js

```
const mongoose = require('mongoose');

const favoriteSchema = new mongoose.Schema({
  userId: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true },
  gameId: { type: mongoose.Schema.Types.ObjectId, ref: 'Game', required: true }
});

module.exports = mongoose.model('Favorite', favoriteSchema);
```

Annexe 7 : Schéma de BDD et Diagramme de classes UML




User
_id : ObjectId
username : String
firstname : String
lastname : String
email : String
password : String ' Haché
telephone : String
role : String // 'user' ou 'admin'
gameTables : ObjectId[]
1
*
creates
creator
participates in
*
*
GameTable
_id : ObjectId
creator : ObjectId
game : ObjectId
startTime : Date
endTime : Date
duration: Number
open : Boolean
participants : ObjectId[]
maxParticipants : Number
*
is associated with
game
1
Game
_id : ObjectId
name : String
type : String
category : String
ageMin : Number
difficulty : String
author : String
playerMin : Number
playerMax : Number
description : String
image : String
partytime : Number
rules : String
resourcesLink : String[]
createdAt : Date
OpeningHours
_id: ObjectId
monday : { startTime: String, endTime: String }
tuesday : { startTime: String, endTime: String }
wednesday : { startTime: String, endTime: String }
thursday : { startTime: String, endTime: String }
friday: { startTime: String, endTime: String }
saturday: { startTime: String, endTime: String }
sunday: { startTime: String, endTime: String }

Annexe 8 : Jeu d’essais

1. Création d'un jeu (Administrateur) :

Scénario : Un administrateur se connecte à l'application et crée un nouveau jeu en remplissant le formulaire de création de jeu avec des données valides.

Résultat attendu :

Un nouveau jeu est créé en base de données, avec les informations fournies.

L'image est copiée dans le dossier public/images du serveur.

L'administrateur est redirigé vers la liste des jeux.

Un message de succès s'affiche.

Résultat obtenu : OK

Analyse des écarts : Aucun écart constaté.

2. Création d'une table de jeu (Utilisateur connecté) :

Scénario : Un utilisateur connecté crée une nouvelle table de jeu en spécifiant une date, une heure, une durée et un nombre maximum de participants valides.

Résultat attendu :

Une nouvelle table de jeu est créée en base de données, associée au jeu et à l'utilisateur connecté.

L'utilisateur est automatiquement inscrit à la table.

L'utilisateur est redirigé vers la page de détails du jeu.

Un message de succès s'affiche.

Résultat obtenu : OK

Analyse des écarts : Aucun écart constaté.

3. Inscription à une table de jeu (Utilisateur connecté) :

Scénario : Un *autre* utilisateur connecté (différent du créateur) tente de s'inscrire à la table de jeu créée précédemment.

Résultat attendu :

L'utilisateur est ajouté à la liste des participants de la table.

Le nombre de participants inscrits est mis à jour.

Un message de succès s'affiche.

Résultat obtenu : OK

Analyse des écarts : Aucun écart constaté.

4. Tentative d'inscription à une table pleine (Utilisateur connecté) :

Scénario : Un utilisateur connecté tente de s'inscrire à une table de jeu qui est pleine (nombre maximum de participants atteint).

Résultat attendu :

Un message d'erreur clair s'affiche, indiquant que la table est pleine.

L'utilisateur *n'est pas* inscrit à la table.

Résultat obtenu : OK

Analyse des écarts : Aucun écart constaté.

5. Création d'une table de jeu avec des paramètres invalides.

Scénario : Un utilisateur tente de créer une table de jeu, mais entre des données non valides (chevauchement de dates, durée de partie inférieure au minimum requis, nombre de participants supérieur au maximum autorisé, etc.).

Résultat attendu : L'application doit afficher un message d'erreur clair et précis, indiquant la nature du problème. La table ne doit *pas* être créée.

Résultat obtenu : OK

Analyse des écarts : Aucun écart constaté.

6. Tentative de suppression de son propre compte (Utilisateur connecté) :

Scénario : Un utilisateur connecté tente de supprimer son propre compte.

Résultat attendu :

Le compte de l'utilisateur est supprimé de la base de données.

L'utilisateur est déconnecté.

Toutes les tables créées par cet utilisateur sont supprimées.

L'utilisateur est retiré de toutes les tables auxquelles il était inscrit.

Un message de confirmation s'affiche.

Résultat obtenu : OK

Analyse des écarts : Aucun écart constaté.

7. Tentative de suppression d'une table par le créateur (Utilisateur connecté, non-admin) :

Scénario : Un utilisateur connecté, créateur d'une table, tente de supprimer cette table.

Résultat attendu :

La table est supprimée de la base de données.

La table est retirée de la liste des tables (après rechargement).

Un message de confirmation s'affiche.

Résultat obtenu : OK

Analyse des écarts : Aucun écart constaté.

8. Tentative d'accès à une page admin par un utilisateur standard :

Scénario : Un utilisateur connecté avec un rôle standard tente d'accéder à une page réservée aux administrateurs directement via l'url.

Résultat attendu : L'utilisateur est redirigé vers la page de connexion et n'a pas accès au contenu de la page.

Résultat obtenu : OK

Analyse des écarts : Aucun écart constaté.

9. Déconnexion :

Scénario : Un utilisateur connecté (standard ou administrateur) clique sur le bouton "Déconnexion".

Résultat attendu :

Le token JWT est supprimé du localStorage.

L'utilisateur est redirigé vers la page de connexion (ou la page d'accueil).

Les informations de l'utilisateur (rôle, ID) sont effacées des BehaviorSubject du AuthService.

Résultat obtenu : OK

Analyse des écarts : Aucun écart constaté.